เปรียบเทียบ Firebase, MongoDB และ Supabase Share

เปรียบเทียบข้อดีข้อเสียของ firebase,mongobase supabase ให้หน่อย

เปรียบเทียบบริการฐานข้อมูลยอดนิยมสามตัวเลือก.
ผมเดาว่า "mongobase" หมายถึง MongoDB (หรือ MongoDB Atlas) นะครับ

ภาพรวม

	Firebase	MongoDB (Atlas)	Supabase
ประเภท	BaaS ของ Google	ฐานข้อมูล NoSQL (document)	BaaS แบบ open source บน PostgreSQL
โครงสร้างข้อมูล	NoSQL (Firestore / Realtime DB)	NoSQL (JSON/BSON document)	SQL (relational)
ฟีเจอร์ครบชุด	ครบ (Auth, Storage, Hosting, Functions)	เน้นฐานข้อมูล ส่วนอื่นต้องหาเพิ่ม	ครบ (Auth, Storage, Edge Functions, Realtime)
Self-host	ไม่ได้	ได้	ได้

Firebase

ข้อดี

- เริ่มต้นเร็วมาก SDK ใช้ง่าย เหมาะกับ mobile (Flutter, Android, iOS)
- Realtime sync และ offline support ทำงานได้ดี
- Auth, Push Notification (FCM), Analytics, Crashlytics อยู่ในระบบเดียวกัน
- Scale ได้อัตโนมัติ ไม่ต้องดูแลเซิร์ฟเวอร์

ข้อเสีย

- Vendor lock-in สูง ย้ายออกยาก
- NoSQL ทำ query ซับซ้อน, join, aggregate ได้จำกัด ต้อง denormalize ข้อมูลเอง
- ค่าใช้จ่ายคิดตามจำนวน read/write คาดเดายาก อาจพุ่งเมื่อมีผู้ใช้มาก
- ต้องเขียน Security Rules ซึ่งหากผิดพลาดจะเสี่ยงด้านความปลอดภัย

MongoDB

ข้อดี

- Schema ยืดหยุ่น เหมาะกับข้อมูลที่โครงสร้างเปลี่ยนบ่อย
- Aggregation pipeline ทรงพลัง ทำ query ซับซ้อนได้ดี
- Scale แนวนอน (sharding) เหมาะกับข้อมูลปริมาณมหาศาล
- ระบบนิเวศใหญ่ มี driver รองรับเกือบทุกภาษา และ self-host ได้

ข้อเสีย

- เป็นแค่ฐานข้อมูล ต้องสร้าง backend, auth, storage เองหรือหา service อื่นมาเสริม
- ความสัมพันธ์ระหว่างข้อมูล (relations) จัดการยากกว่า SQL
- ควรออกแบบ schema และ index ให้ดี ไม่เช่นนั้นประสิทธิภาพตก
- ไม่มี realtime sync สำเร็จรูปแบบ Firebase

Supabase

ข้อดี

- ใช้ PostgreSQL จริง รองรับ SQL, join, transaction, extension ต่างๆ (เช่น pgvector สำหรับงาน AI)
- Open source และ self-host ได้ ลดความเสี่ยงเรื่อง lock-in
- มี Auth, Storage, Realtime, Edge Functions ครบ และสร้าง REST/GraphQL API ให้อัตโนมัติ
- Row Level Security ควบคุมสิทธิ์ที่ระดับฐานข้อมูลได้ละเอียด

ข้อเสีย

- อายุน้อยกว่า Firebase ระบบนิเวศและเอกสารยังเล็กกว่า
- Offline support ไม่ดีเท่า Firebase
- ต้องเข้าใจ SQL และ RLS พอสมควร มี learning curve
- การ scale ขนาดใหญ่มากต้องเข้าใจเรื่อง Postgres tuning เอง

เลือกอะไรดี

- **Firebase:** เหมาะกับ prototype เร็วๆ, แอปมือถือ, แอปที่ต้องการ realtime/offline เช่น แชท
- **MongoDB:** เหมาะกับข้อมูลโครงสร้างไม่แน่นอนหรือปริมาณมาก และมีทีมที่เขียน backend เองอยู่แล้ว
- **Supabase:** เหมาะกับแอปที่ข้อมูลมีความสัมพันธ์กัน (เช่น ระบบสมาชิก, e-commerce, dashboard) และอยากได้ความสะดวกแบบ BaaS โดยไม่ผูกติดกับผู้ให้บริการ

ถ้าเป็นโปรเจกต์ทั่วไปที่ยังไม่แน่ใจ ผมมักแนะนำ Supabase เป็นจุดเริ่มต้น เพราะ SQL ยืดหยุ่นและย้ายออกง่าย ส่วนราคาและโควตาฟรีของแต่ละเจ้าเปลี่ยนบ่อย ควรเช็กหน้า pricing ล่าสุดก่อนตัดสินใจครับ

ถ้าบอกได้ว่าจะใช้กับโปรเจกต์แบบไหน (เช่น เว็บ, มือถือ, IoT) ผมช่วยเลือกให้เจาะจงขึ้นได้ครับ

มี database นอกเหนือจากนี้มั้ย

มีเยอะมากครับ แบ่งตามประเภทให้ดูง่ายๆ

BaaS (ครบชุดแบบ Firebase/Supabase)

- **Appwrite:** open source, self-host ได้ มี Auth, Storage, Functions ครบ ใช้ง่าย
- **PocketBase:** ไฟล์เดียวจบ (Go + SQLite) เบามาก เหมาะกับโปรเจกต์เล็กถึงกลาง และบอร์ดอย่าง Raspberry Pi
- **Nhost:** คล้าย Supabase แต่ใช้ GraphQL (Hasura) เป็นหลัก
- **AWS Amplify:** ของ Amazon ต่อกับ DynamoDB/Cognito

SQL (Relational)

- **PostgreSQL:** มาตรฐานยอดนิยม ฟีเจอร์เยอะ
- **MySQL / MariaDB:** ใช้กันแพร่หลาย โดยเฉพาะ PHP/WordPress
- **SQLite:** ฐานข้อมูลในไฟล์เดียว ไม่ต้องมีเซิร์ฟเวอร์ เหมาะกับแอปเล็กและอุปกรณ์ฝังตัว
- **PlanetScale / Neon / Turso:** บริการ managed แบบ serverless (MySQL, Postgres, SQLite ตามลำดับ)
- **CockroachDB:** SQL แบบกระจายหลายภูมิภาค scale ง่าย

NoSQL

- **Redis:** เก็บใน memory เร็วมาก ใช้ทำ cache, session, queue
- **DynamoDB:** key-value/document ของ AWS scale ได้มหาศาล
- **Cassandra / ScyllaDB:** เหมาะกับข้อมูลเขียนหนักปริมาณมาก
- **CouchDB:** เน้น sync และ offline-first
- **Neo4j:** Graph database สำหรับข้อมูลที่เชื่อมโยงกันมาก เช่น social network

Time-series (เหมาะกับข้อมูลเซนเซอร์/IoT)

- **InfluxDB**
- **TimescaleDB:** ส่วนขยายของ PostgreSQL
- **Prometheus:** เน้น monitoring

Vector Database (งาน AI/embedding)

- **Pinecone, Qdrant, Weaviate, Chroma**
- หรือใช้ **pgvector** บน PostgreSQL/Supabase ก็ได้

ที่น่าสนใจเป็นพิเศษ

- ถ้าอยากได้ **ง่ายและฟรี:** PocketBase หรือ Appwrite
- ถ้าทำ **IoT/เซนเซอร์:** InfluxDB หรือ TimescaleDB
- ถ้าทำแอป **offline-first:** CouchDB หรือ Firebase
- ถ้าต้องการ **เบาที่สุด:** SQLite

บอกประเภทโปรเจกต์ที่จะทำได้เลยครับ แล้วผมจะช่วยแนะนำตัวที่เหมาะที่สุดให้

ทำเป็นแบบ https://makergo.app/ MyPOS เหมาะกับแบบไหนดี

Fetched: MakerGO https://makergo.app/

จากหน้าเว็บ MakerGO เป็นระบบ POS สำหรับบูธงานอาร์ตแฟร์ ใช้บน iPad และ**รองรับ offline** ซึ่งเป็นข้อกำหนดที่สำคัญที่สุดในการเลือก database เพราะงานแบบนี้อินเทอร์เน็ตในงานมักไม่เสถียร แต่ต้องขายของและบันทึกยอดได้ตลอด

หลักการออกแบบ

ให้ข้อมูลเก็บในเครื่อง (IndexedDB/SQLite) ก่อน แล้วค่อย sync ขึ้นคลาวด์เมื่อมีเน็ต การเรียก API ตรงๆ ทุกครั้งที่ขายของจะไม่เวิร์กเมื่อเน็ตหลุด

ตัวเลือกที่เหมาะ

1. Firebase (Firestore) เหมาะที่สุดถ้าอยากทำให้เสร็จเร็ว

- มี offline persistence ในตัว เปิดใช้ได้เลย ข้อมูลจะ sync เองเมื่อเน็ตกลับมา
- Auth, Hosting อยู่ครบ
- ข้อเสียคือ query ซับซ้อนทำยาก เช่น รายงานยอดขายรายสินค้า รายวัน และราคาคิดตามจำนวน read/write

2. Supabase + local-first sync library เหมาะถ้าอยากได้ SQL และรายงานที่ดี

- ข้อมูลขาย สินค้า สต็อก เป็นข้อมูลเชิงสัมพันธ์ เหมาะกับ PostgreSQL
- Supabase เองไม่มี offline ในตัว ต้องเสริมด้วย PowerSync, RxDB หรือ Dexie (ฝั่ง client) แล้วเขียน sync เอง
- ได้ความยืดหยุ่นด้านรายงานและไม่ผูกกับผู้ให้บริการ แต่ใช้เวลาพัฒนามากกว่า

3. CouchDB + PouchDB เหมาะถ้าเน้น offline sync โดยเฉพาะ

- ออกแบบมาเพื่อ sync สองทางแบบ offline-first ตั้งแต่ต้น
- ข้อเสียคือ query/รายงานไม่ถนัด และ ecosystem เล็กลง

4. PocketBase + IndexedDB เหมาะกับโปรเจกต์เล็กหรือเริ่มต้น

- เบา self-host ง่าย แต่ offline sync ต้องเขียนเอง

ที่ผมแนะนำ

- **ทำเร็ว/ส่งงาน/prototype:** Firebase Firestore เพราะ offline ใช้ได้ทันที
- **อยากทำจริงจังและมีรายงานยอดขาย:** Supabase + RxDB หรือ PowerSync ซึ่งเป็นแนวทางเดียวกับที่ระบบ POS แบบนี้มักใช้

ส่วนอื่นที่ต้องมี

- ทำเป็น PWA (Service Worker) เพื่อให้เปิดใช้บน iPad ได้โดยไม่ต้องลง App Store
- ใช้ UUID เป็น ID ของออเดอร์ที่สร้างฝั่ง client เพื่อไม่ให้ชนกันตอน sync จากหลายเครื่อง
- เก็บสต็อกเป็นรายการเคลื่อนไหว (ขายเข้า/ออก) แทนการเขียนทับตัวเลข จะช่วยลดปัญหาข้อมูลชนกันตอน sync

ถ้าบอกได้ว่าจะทำคนเดียวหรือเป็นทีม ใช้กี่เครื่อง และต้องมีรายงานหรือระบบสต็อกลึกแค่ไหน ผมจะช่วยเลือกและวางโครงสร้างตารางให้ได้เลยครับ

firebase มีบริการใช้ฟรีมั้ย หากแอพหรือเว็บที่ไว้ใช้พัฒณามาเพื่อใช้เอง

Searched the web Firebase pricing Spark plan free quota Firestore 2026 ▸

มีครับ Firebase มีแผนฟรีชื่อ **Spark** ไม่ต้องใส่บัตรเครดิต และไม่มีทางเกิดบิล เพราะเมื่อบริการใดใช้โควตาหมด บริการนั้นจะหยุดตอบสนองจนกว่าจะรีเซ็ต จึงเหมาะกับแอปที่ทำไว้ใช้เอง dev

โควตาฟรีหลักๆ (Spark)

- **Firestore:** เก็บข้อมูลได้ 1 GiB, อ่าน 50,000 ครั้ง/วัน, เขียน 20,000 ครั้ง/วัน, ลบ 20,000 ครั้ง/วัน costbench
- **Authentication:** อีเมลและ social login ได้ถึง 50,000 MAU supertokens
- **Cloud Functions:** 2 ล้านครั้งเรียกใช้ต่อเดือน (แต่ถ้าต้องการ Functions จริงๆ มักต้องอัปเกรด Blaze ตามเงื่อนไขของ Google Cloud ควรเช็กหน้าทางการ) dev
- **Hosting:** ใช้ฟรีได้ในแผน Spark พร้อม SSL
- **Cloud Messaging และ Crashlytics:** เป็นบริการที่ไม่มีค่าใช้จ่าย verdent

ข้อควรระวัง

- **Cloud Storage (เก็บรูป/ไฟล์) ต้องใช้แผน Blaze แล้ว** ตั้งแต่ 3 กุมภาพันธ์ 2026 แม้ใช้ในปริมาณน้อยมาก ถ้า POS ของคุณต้องเก็บรูปสินค้า อาจต้องหาทางอื่น เช่น เก็บเป็นรูปขนาดเล็กใน Firestore, ใช้ Supabase Storage หรือ Cloudinary แทน dev
- **Phone Auth (SMS) ไม่ฟรี** ต้องใช้ Blaze และคิดเงินต่อข้อความ ถ้าใช้เองใช้ล็อกอินด้วยอีเมลหรือ Google แทนได้ supertokens
- **Blaze ไม่ได้แปลว่าต้องจ่ายเสมอ** แผนนี้ยังรวมโควตาฟรีเดิมไว้ และคิดเงินเฉพาะส่วนที่เกิน แต่ต้องผูกบัตร และการแจ้งเตือนงบประมาณ (50%/80%/100%) จะแค่แจ้งเตือน ไม่ได้หยุดการอ่านข้อมูลให้ ถ้าอัปเกรดควรตั้ง alert ไว้ verdent

ใช้เองพอไหม

ใช้เองคนเดียวหรือทีมเล็ก ถ้าเป็นระบบ POS ขายของหน้าบูธ ปกติไม่น่าถึง 20,000 writes/วัน ดังนั้น **Firestore + Auth + Hosting บน Spark เพียงพอแน่นอน** ทิ้งไว้ก่อนได้เลย เคล็ดลับคืออย่าดึงข้อมูลทั้ง collection ทุกครั้ง เพราะจะกิน read quota เร็ว (เปิด offline persistence ก็ช่วยลด read ได้ด้วย)

โควตาและเงื่อนไขอาจเปลี่ยนได้ ควรเช็กหน้า Firebase pricing ทางการก่อนเริ่มใช้งานจริงครับ

do research about POS system สำหรับบูธงานอาร์ตแฟร์ใช้บน iPad และรองรับ offline. what important function it need inside applicatio. I have idea from
-loyverse pos
-MakerGO
-BoothMate

Searched the web, read memory, read a page ▸

สิ่งที่ค้นเจอ

- **Loyverse:** มีเอกสารทางการเรื่อง offline ละเอียด ใช้เป็นตัวอย่างว่า POS ทั่วไปทำอะไรได้/ไม่ได้ตอนออฟไลน์
- **GoodsBar:** ไม่ใช่ตัวที่คุณระบุ แต่เป็นแอปที่ตรงกับโจทย์คุณที่สุด (iPad POS สำหรับบูธ artist alley ทำงาน offline)

- MakerGO: เนื้อหาหน้าเว็บโหลดผ่านเครื่องมือไม่ได้ เห็นแค่คำอธิบายว่าเป็น POS ธีมพาสเทลสำหรับบูธงานอาร์ตแฟร์ ปรับให้เหมาะกับ iPad และรองรับ offline ฟีเจอร์ภายในต้องลองเปิดดูเองครับ makergo
- BoothMate: หาข้อมูลไม่เจอ ผมจึงไม่เดาฟีเจอร์ให้ ถ้ามีลิงก์หรือสกรีนช็อตส่งมาได้ครับ

บทเรียนจาก Loyverse (ข้อจำกัดตอน offline)

ขายและทำ shift ได้ตอนออฟไลน์ ใบเสร็จเก็บในเครื่องและขึ้นสถานะ “Unsynced” แล้ว sync เองเมื่อมีเน็ต แต่ทำ refund ไม่ได้ ลงทะเบียนลูกค้าใหม่ไม่ได้ ไม่แสดงสต็อกสินค้า และเครื่องรูดบัตรไม่ทำงาน นี่คือช่องว่างที่แอปของคุณทำให้ดีกว่าได้ เพราะหน้าบูธเน็ตหลุดบ่อยและต้องเห็นสต็อกตลอด loyverse loyverse

จุดเด่นของ GoodsBar (เหมาะเป็นต้นแบบ)

ทุกออเดอร์บันทึกในเครื่อง ไม่ต้องใช้อินเทอร์เน็ต ผู้ช่วยขายต่อผ่าน hotspot หรือ Wi-Fi วงเดียวกันได้ แยกยอดขายตามครีเอเตอร์เมื่อแชร์โต๊ะ มีหลายสกุลเงิน พิมพ์ใบเสร็จดีไซน์เอง สต็อกแยกตามอีเวนต์ มีโปรโมชันของแถมอัตโนมัติ สแกนบาร์โค้ดด้วยกล้อง และดูกำไรต่อออเดอร์ ราคาแผน Master $6.99/เดือน หรือ $69.99/ปี apple apple

ฟีเจอร์ที่ควรมี แบ่งตามความสำคัญ

Must-have (MVP)

1. **หน้าขาย:** ตารางสินค้าแบบรูปภาพใหญ่ กดเพิ่มลงตะกร้า ปรับจำนวน เหมาะนิ้วสัมผัสบน iPad
2. **Offline-first จริง:** บันทึกลงเครื่องก่อนเสมอ มีสถานะ synced/unsynced และ sync อัตโนมัติ
3. **จัดการสินค้า:** ชื่อ รูป ราคา หมวดหมู่ ต้นทุน และตัวเลือก (variant) เช่น ขนาด/ลาย
4. **สต็อก:** ตัดสต็อกตอนขาย แสดงของใกล้หมด/หมด (ส่วนนี้ Loyverse ทำไม่ได้ตอน offline)
5. **วิธีชำระเงิน:** เงินสด (ปุ่มธนบัตรด่วน คำนวณเงินทอน), QR PromptPay, โอน และบันทึกประเภทการชำระเงิน
6. **ส่วนลด:** ลดเป็น % หรือบาท ทั้งรายการและทั้งบิล
7. **ประวัติออเดอร์ + ยกเลิก/คืนเงิน** ทำได้ตอน offline (บันทึกเป็นรายการแก้ไข ไม่ลบของเดิม)
8. **สรุปยอดปิดวัน/ปิดงาน:** ยอดรวมแยกตามช่องทางชำระ เทียบเงินสดที่นับได้ (Loyverse มีฟีเจอร์ shift คล้ายกันเพื่อดูเงินขาด/เกินตอนสิ้นสุดกะ) loyverse

Should-have (สำคัญสำหรับงานอีเวนต์)

9. **Event/งาน:** สร้างงานแยก (เช่น “Comic Con มี.ค.”) แต่ละงานมีสต็อกเริ่มต้นและรายงานของตัวเอง
10. **Bundle/เซ็ต** เช่น ซื้อ 3 แถม 1, ราคาเซ็ต และโปรโมชันแบบ spend-and-get
11. **ต้นทุนและกำไร:** ต้นทุนสินค้า ค่าบูธ ค่าใช้จ่ายอื่น คำนวณกำไรสุทธิต่องาน
12. **รายงาน:** สินค้าขายดี ยอดรายชั่วโมง (รู้ช่วง peak) เปรียบเทียบงานต่องาน
13. **Export CSV** และ backup/restore ข้อมูล
14. **PIN/สิทธิ์:** จำกัดการแก้ราคา/ลบออเดอร์สำหรับผู้ช่วยขาย

Nice-to-have

15. สแกนบาร์โค้ด/QR ด้วยกล้อง, พิมพ์ใบเสร็จและฉลาก (Bluetooth printer)
16. หลายเครื่องในบูธเดียวกัน (sync ผ่าน Wi-Fi วงเดียวกัน) และแยกยอดตามคนขาย/ศิลปินที่แชร์โต๊ะ
17. ใบเสร็จ/e-receipt ที่ใส่โลโก้และ QR โซเชียล
18. ธีมสีและหน้าจอ standby
19. Preorder/รับออเดอร์สำหรับจัดส่งทีหลัง

ข้อควรออกแบบเป็นพิเศษ

- ID ออเดอร์สร้างฝั่งเครื่อง (UUID) กันชนกันเมื่อ sync หลายเครื่อง
- **สต็อกเป็นรายการเคลื่อนไหว** ไม่เขียนทับตัวเลข ช่วยแก้ปัญหา conflict ตอน sync
- **ข้อมูลทางการเงินห้ามหาย:** ทำ backup อัตโนมัติและปุ่ม export ที่หน้าปิดงาน เพราะข้อมูลเก็บในเครื่องเสี่ยงหาย ถ้าใช้ PWA บน iOS ข้อมูลใน browser อาจถูกล้างได้ จึงควร sync ขึ้นคลาวด์เมื่อมีเน็ต
- **การรูดบัตร** แทบทำ offline ไม่ได้ในทุกระบบ ในไทยเน้น QR PromptPay (แสดงรูป QR ของร้านเอง แล้วให้คนขายกดยืนยัน) จะง่ายกว่า

ลำดับการพัฒนาที่แนะนำ

ข้อ 1–8 ก่อน เป็น MVP ที่ใช้งานจริงได้ จากนั้นเพิ่มระบบ Event (ข้อ 9) และ bundle/โปรโมชัน เพราะเป็นจุดที่ POS ทั่วไปอย่าง Loyverse ไม่ได้ออกแบบมาเพื่องานบูธ

ถ้าต้องการ ผมช่วยออกแบบโครงสร้างตาราง (products, events, orders, stock movements) หรือวาด flow หน้าจอหลักให้ต่อได้เลยครับ



loyverse-pos-manual-th.pdf

PDF

นี้คือ manual จากแอพต่างๆเป็นข้อมูลเพิ่มเติม. ช่วยสรุปการตัดสินใจ และ funtion ที่ควรมีใน app ให้หน่อย

Ran 3 commands, read a file ▸

ผมอ่าน manual ของ MakerGO ครบแล้ว ส่วน Loyverse มี 350 หน้า ผมดูสารบัญทั้งหมดและเจาะอ่านส่วนที่เกี่ยวกับ offline เป็นหลัก สรุปจึงอิงจากส่วนนี้ครับ

ข้อสังเกตจาก manual ทั้งสอง

MakerGO ออกแบบมาตรงโจทย์บูธอาร์ตแฟร์มาก

- เป็นเว็บแอปเปิดผ่านเบราว์เซอร์ ไม่ต้องลงแอป
- ล็อกอินด้วย Google และเชิญทีมผ่านลิงก์อายุ 7 วัน ใช้ได้ครั้งเดียว
- แยกสินค้าตาม "ศิลปิน" ใช้ได้เมื่อหลายคนแชร์บูธ มีส่วนลดแยกตามศิลปิน และ export Excel แยกชีตตามศิลปิน
- มีระบบ "อีเวนต์ที่กำลังใช้งาน" ที่ซ่อนสินค้าของศิลปินที่ไม่ได้มางานนั้นจากหน้าขายอัตโนมัติ
- ชำระด้วยเงินสด (ปุ่มลัดธนบัตร คำนวณเงินทอน) และ QR พร้อมเพย์

Loyverse เป็น POS ทั่วไป ฟีเจอร์เยอะมากแต่ส่วนใหญ่ไม่จำเป็นกับบูธ เช่น ใบสั่งซื้อ ซัพพลายเออร์ การผลิต ครัว ระบบพนักงานและเวลาทำงาน หลายสาขา สิ่งที่เอามาใช้ได้ คือ shift (ปิดกะนับเงินสด), variant, สแกนบาร์โค้ดด้วยกล้อง, แจ้งเตือนสต็อกต่ำ, พักบิล, ใบเสร็จและพิมพ์ใบเสร็จ, PIN และสิทธิ์พนักงาน, export ข้อมูล

จุดอ่อนของทั้งคู่ ซึ่งคุณทำให้ดีกว่าได้

จุดอ่อน	ที่มา	แนวทางแก้
ออกจากระบบตอน offline แล้วยอดขายที่ยังไม่ sync หายหมด	Loyverse (manual เตือนเอง)	บล็อกการ logout ถ้ายังมีรายการ unsynced และแสดงจำนวนที่ค้างให้เห็นตลอด
ปิดแอปก่อน sync อาจเสียข้อมูล	MakerGO	เก็บใน IndexedDB ที่คงอยู่ถาวร และทำ sync ซ้ำอัตโนมัติ
Refund ทำไม่ได้ตอน offline, ไม่เห็นสต็อกตอน offline	Loyverse	ทำเป็น local-first เต็มรูปแบบ ทั้งสต็อกและ void
Void กับลบสินค้า ย้อนกลับไม่ได้	MakerGO	ใช้ soft delete, void เป็นรายการแก้ไข และเก็บ audit log
ไม่ตรวจยอดเงินโอนเข้า ต้องเช็กแอปธนาคารเอง	MakerGO	ทำปุ่มยืนยันที่ชัดเจน บันทึกว่าใครยืนยัน (การเช็กอัตโนมัติต้องใช้ payment gateway ซึ่งเกินขอบเขตเบื้องต้น)
รูปสินค้าไม่แสดงตอน offline ถ้าไม่เคยโหลด	MakerGO	cache รูปทั้งหมดล่วงหน้า (Service Worker) ตอน sync ครั้งแรก
สินค้าไม่มี variant (ขนาด/ลาย)	MakerGO	ถ้าต้องการ ให้เพิ่มใน v2

การตัดสินใจที่สรุปได้

1. **แพลตฟอร์ม:** ทำเป็น PWA เหมือน MakerGO ใช้ได้บน iPad และคอมพิวเตอร์ ไม่ต้องผ่าน App Store
2. **ข้อมูล:** local-first (เก็บในเครื่องก่อน แล้ว sync) ตามที่คุยกันก่อนหน้า ตัวเลือกคือ Firebase Firestore (ง่ายสุด offline ในตัว) หรือ Supabase + ไลบรารี sync (รายงานดีกว่า แต่ทำเองมากกว่า)
3. **ล็อกอิน:** Google login + ลิงก์เชิญทีม และมี 2 บทบาท (เจ้าของ/ผู้ช่วยขาย)
4. **ชำระเงิน:** เงินสด + พร้อมเพย์ ส่วนบัตรเครดิตตัดออก เพราะใช้ offline ไม่ได้ทุกระบบ ข้อควรรู้คือ QR พร้อมเพย์แบบระบุยอดสร้างฝั่งเครื่องได้โดยไม่ต้องใช้เน็ต (มีไลบรารีสำเร็จรูป) ลูกค้าจึงไม่ต้องกรอกยอดเอง
5. **หัวใจของโมเดลข้อมูล:** สินค้า → ศิลปิน/หมวดหมู่ → **อีเวนต์** (สต็อกและรายงานแยกตามงาน) → ออเดอร์ (UUID สร้างฝั่งเครื่อง) → รายการเคลื่อนไหวสต็อก

ฟีเจอร์ที่ควรมี

v1 (MVP ที่ใช้งานจริงได้)

- หน้าขาย: ตารางสินค้าแบบรูปใหญ่ ค้นหา กรองหมวดหมู่ ตะกร้า และแสดงจำนวนคงเหลือบนการ์ด
- ชำระเงินสด (ปุ่มลัดธนบัตร คำนวณเงินทอน) และ QR พร้อมเพย์
- ส่วนลด ทั้งระดับรายการและทั้งบิล
- คลังสินค้า: เพิ่ม/แก้/คัดลอกสินค้า ชื่อ รูป ราคา **ต้นทุน** หมวดหมู่ สต็อก
- ตัดสต็อกอัตโนมัติ พร้อมป้ายสินค้าหมดและของใกล้หมด
- Offline-first พร้อมแถบสถานะ sync (กี่รายการค้างอยู่) และบล็อก logout ถ้ามีรายการค้าง
- ประวัติการขาย และ void ได้ (คืนสต็อก)
- รายงาน: ยอดวันนี้ รายได้รวม จำนวนบิล สินค้าขายดี และกราฟยอดรายชั่วโมง
- ปิดงาน/ปิดกะ: สรุปยอดแยกเงินสดกับ QR และเทียบเงินสดที่นับได้

- Export Excel/CSV และ import CSV แบบมีหน้า preview ก่อนบันทึก

v1.5 (ทำให้เป็นระบบสำหรับงานอีเวนต์)

- **อีเวนต์:** สร้างงาน กำหนดว่าศิลปิน/สินค้าไหนเข้าร่วม และเลือก "อีเวนต์ที่กำลังใช้งาน" เพื่อกรองหน้าขาย
- สต็อกเริ่มต้นต่องาน และรายงานเปรียบเทียบงานต่องาน
- **ศิลปิน** (ถ้าแชร์บูธ): ยอดแยกตามศิลปิน ส่วนลดแยกตามศิลปิน export แยกชีต
- กำไรต่อออเดอร์และกำไรสุทธิต่องาน (รวมค่าบูธ/ค่าใช้จ่าย)
- เชิญผู้ช่วยขายผ่านลิงก์ พร้อมสิทธิ์จำกัด
- พักบิล และแจ้งเตือนสต็อกต่ำ

v2 (ค่อยเพิ่ม)

- Variant (ขนาด/ลาย) และ bundle/ราคาเซ็ต หรือซื้อครบแถม
- สแกนบาร์โค้ดด้วยกล้อง
- พิมพ์ใบเสร็จ Bluetooth และใบเสร็จที่ใส่โลโก้ + QR โซเชียล
- หลายเครื่องในบูธเดียวกัน (sync ผ่านเครือข่ายเดียวกัน)
- PIN สำหรับการแก้ราคา/void
- Preorder สำหรับของที่ไม่มีหน้างาน

ตัดทิ้งก่อน: ใบสั่งซื้อ/ซัพพลายเออร์ ระบบลูกค้าสะสมแต้ม ภาษี หลายสาขา จอลูกค้า ครัว (เกินความจำเป็นของบูธ)

ลำดับการสร้างที่แนะนำ

1. โมเดลข้อมูล + local DB + หน้าขายเงินสด (ได้ใช้งาน offline ได้เลย)
2. คลังสินค้า + ตัดสต็อก + void
3. พร้อมเพย์ + ปิดงาน + รายงาน
4. sync ขึ้นคลาวด์ + ล็อกอิน + เชิญทีม
5. อีเวนต์ + ศิลปิน

คำถามที่จะตัดสินว่าต้องทำระบบ "ศิลปิน" ตั้งแต่ต้นหรือไม่: คุณจะใช้ขายคนเดียว หรือแชร์บูธกับเพื่อนศิลปินหลายคนครับ ถ้าตอบได้ ผมจะช่วยร่างโครงสร้างตารางและ flow หน้าจอหลักให้ต่อได้เลยครับ

Create a Claude Doc from your last answer.

Make doc changes (batch) ▸

สร้างเอกสารเสร็จแล้วครับ อ่านดู แก้ไขในเอกสารได้เลย หรือคอมเมนต์ไว้แล้วผมจะช่วยปรับให้ ถ้าต้องการไฟล์ Word หรือ PDF ก็ดาวน์โหลดจากเอกสารได้ครับ

จากที่สรุปมาดีมากเข้าใจง่าย แต่ถ้าหากอยากมาพัฒนาให้รองรับหลายคนอย่างเช่น 1 บูธแต่เราต้องออกร่วมกับคนอื่นจะต้องออกแบบยังไง

อัปเดตความจำแล้ว booth-pos-app.md ▸

Continue

ออกแบบระบบให้รองรับหลายศิลปินแชร์บูธเดียวกัน

MakerGO ทำโมเดลนี้ไว้แล้วในระดับหนึ่ง (ดูจาก manual) ผมจะขยายให้ครบและชี้จุดที่ต้องระวังเพิ่มเติม

แนวคิดหลัก: แยก "ร้าน/บูธ" ออกจาก "เจ้าของสินค้า"

ต้องคิดเป็น 2 ชั้นไม่ปนกัน:

- Shop/Booth = หน่วยที่ขายของจริงหน้างาน มี 1 เครื่อง (หรือหลายเครื่อง) ขาย
- Artist/Vendor = เจ้าของสินค้าแต่ละคนที่เอาของมาวางขายในบูธนั้น

สินค้าทุกชิ้นต้อง**ผูกกับ artist_id เสมอ** ไม่ใช่แค่ผูกกับร้าน เพราะตอนสรุปเงินปลายงานต้องแยกจ่ายคืนให้แต่ละคนตามยอดขายจริง

โครงสร้างตารางที่แนะนำ

```
Shop (บูธ)
 └─ id, name, owner_user_id, promptpay_id

Member (คนในบูธ)
 └─ id, shop_id, user_id, role (owner/member), commission_rate (ถ้ามี)

Artist (เจ้าของสินค้า — อาจไม่ได้ล็อกอินเองก็ได้)
 └─ id, shop_id, name, linked_user_id (nullable — เผื่อศิลปินไม่ได้ขายเอง แค่ฝากของ)

Product
 └─ id, shop_id, artist_id, name, price, cost, stock, category

Order
 └─ id, shop_id, created_by_user_id, total, payment_method, created_at

OrderItem
 └─ id, order_id, product_id, artist_id (denormalize ตอนขาย กันกรณีสินค้าถูกลบทีหลัง),
    qty, price_at_sale, discount_amount

Settlement (ปิดงาน/เคลียร์เงินแต่ละศิลปิน)
 └─ id, shop_id, event_id, artist_id, gross_sales, commission, net_payout, paid
(bool)
```

จุดสำคัญ: `artist_id` **ใน OrderItem ต้อง snapshot ไว้ตอนขาย** ไม่ใช่ join จาก Product ตอนทำรายงาน เพราะถ้าสินค้าถูกแก้ไขหรือลบ เจ้าของสินค้าเปลี่ยน ข้อมูลยอดขายเก่าต้องไม่ขยับตาม

ฟีเจอร์ที่ต้องเพิ่มจาก MVP เดิม

1. ตะกร้าแบบ multi-artist

เมื่อลูกค้าซื้อของจากหลายศิลปินในบิลเดียว ระบบต้อง:

- แสดงยอดรวมทั้งบิลให้ลูกค้าจ่ายครั้งเดียว (เงินสด/QR ใบเดียว ไม่แยกจ่าย)
- แต่เบื้องหลังแตกยอดเป็นรายศิลปินอัตโนมัติ
- **ส่วนลดต้องเลือกได้ว่าลดของศิลปินคนไหน** (ซึ่ง MakerGO ทำไว้แล้ว) — ถ้าลดทั้งบิล ต้องมีกติกาว่าแบ่งสัดส่วนส่วนลดยังไง (เช่น ลดตามสัดส่วนมูลค่าของแต่ละคนในบิล)

2. ระบบค่าคอมมิชชัน/ค่าเช่าที่

ถ้ามีการหักเปอร์เซ็นต์ค่าที่บูธจากยอดขายแต่ละคน (เรื่องปกติเวลาแชร์บูธ) ต้องมี:

- ตั้งค่า commission_rate ต่อศิลปิน (อาจไม่เท่ากัน)
- รายงานสรุป: ยอดขายรวม − ค่าคอม = ยอดที่ต้องจ่ายคืนศิลปินแต่ละคน
- ปุ่ม "บันทึกว่าจ่ายแล้ว" ต่อคน กันลืมหรือจ่ายซ้ำ

3. สิทธิ์การมองเห็นข้อมูล

ต้องตัดสินใจเรื่องนี้ให้ชัดตั้งแต่ต้น เพราะกระทบ query และ UI ทั้งระบบ:

- ศิลปินแต่ละคนเห็นแค่ยอดขายของตัวเอง หรือเห็นยอดรวมทั้งบูธได้ด้วย?
- ถ้าเป็นแบบ MakerGO (role: owner/member) — member เห็นได้หมดทุกคน จะง่ายกว่าแต่ลดความเป็นส่วนตัว
- ถ้าต้องการความเป็นส่วนตัวมากกว่า ต้องเพิ่ม role ที่ 3 เช่น “artist” ที่ query ถูกกรองด้วย artist_id = ตัวเอง เท่านั้น

4. การ void/คืนสินค้าข้ามศิลปิน

ถ้าบิลเดียวมีของหลายคน แล้วลูกค้าคืนแค่ชิ้นเดียว ระบบต้อง void ได้แบบ**รายชิ้น** ไม่ใช่ทั้งบิล (MakerGO ทำรายบิลตาม manual ที่อ่านมา ถ้าจะทำดีกว่านี้ต้องรองรับ partial void)

5. CSV import ต้องรู้จัก artist เป็น field บังคับ

ตามที่ MakerGO ทำไว้ (name + artist ใช้ match ตอน import) — ถ้าจะทำต่อ ให้คง behavior นี้เพราะกันชื่อสินค้าซ้ำระหว่างศิลปินสองคน

ของที่ต้องคิดเผื่อ (ง่ายกว่าถ้าคิดตอนนี้ ยากกว่ามากถ้าคิดทีหลัง)

- **ศิลปินไม่เข้าร่วมทุกงาน:** เหมือนที่ MakerGO มี "อีเวนต์" ที่เลือกว่าใครมางานไหน — เก็บเป็นความสัมพันธ์ many-to-many ระหว่าง Event กับ Artist
- **ศิลปินออกจากทีมกลางงาน:** ถ้าลบ Artist ออก ประวัติการขายเก่าต้องไม่หาย (ใช้ snapshot ใน OrderItem ตามข้อบนจะช่วยตรงนี้)
- **สกุลเงิน/ราคาทุนเป็นความลับของแต่ละศิลปิน:** บางคนอาจไม่อยากให้เพื่อนในบูธเห็นต้นทุนของตัวเอง ควรแยกสิทธิ์การเห็น field `cost` ออกจาก field อื่น

ลำดับที่แนะนำถ้าจะทำ

1. เพิ่ม `artist_id` ลงทุกตารางที่เกี่ยวข้อง (Product, OrderItem) ตั้งแต่แรก แม้ตอนนี้จะใช้คนเดียวก็ตาม — ย้อนกลับมาเพิ่มทีหลังยากกว่าทำตั้งแต่ต้นมาก
2. ทำตะกร้า multi-artist + ส่วนลดแยกตามคน
3. ทำรายงานแยกตามศิลปิน + commission + settlement
4. ทำ role “artist” ถ้าต้องการความเป็นส่วนตัว
5. ค่อยทำ partial void และ permission ละเอียดอื่นๆ

ถ้าอยากได้ schema แบบ SQL จริงๆ (CREATE TABLE) พร้อม foreign key หรืออยากให้ร่าง flow หน้าจอ "เลือกศิลปินในตะกร้า" ให้ดู บอกได้เลยครับ

> ดีงั้นช่วยสรุปเอาไปสานต่อให้ antigravity ide ไปทำต่อ โดยสรุปจากแชทที่เราคุยกันโดยเน้น function ที่จำเป็นต้องมีก่อน version 1.0 อย่าลืมว่าเป็นแอพพลิเคชั่นแล้ว

Read and updated memory ▸

Dev Brief — Booth POS App v1.0 (สำหรับส่งต่อให้ Antigravity IDE)

พร้อมก็อปทั้งก้อนนี้ไปวางให้ Antigravity ได้เลยครับ เขียนให้ชัดระดับที่ AI coding agent เอาไปเริ่มลงมือได้ทันที

1. ภาพรวมโปรเจกต์

สร้าง **แอปพลิเคชัน** POS (ไม่ใช่แค่เว็บเพจธรรมดา) สำหรับขายสินค้าหน้าบูธงานอาร์ตแฟร์ ใช้งานบน **iPad เป็นหลัก** ต้องทำงานได้แม้ไม่มีอินเทอร์เน็ต (เน็ตหน้างานไม่เสถียร) แรงบันดาลใจจาก Loyverse POS, MakerGO, BoothMate

สิ่งสำคัญที่สุดของโปรเจกต์นี้คือ offline-first — ไม่ใช่แอปที่เรียก API ทุกครั้งที่กดปุ่ม แต่บันทึกลงเครื่องก่อนเสมอ แล้วค่อย sync ทีหลัง

2. แพลตฟอร์มและสถาปัตยกรรม

- **ทำเป็น** PWA (Progressive Web App) — ติดตั้งหน้าจอ Home บน iPad ได้ ทำงานแบบแอปจริง ไม่ต้องผ่าน App Store
- ต้องมี **Service Worker** สำหรับ cache ไฟล์แอปทั้งหมด (JS/CSS/รูปสินค้า) ให้เปิดใช้ได้แม้ปิดเน็ตสนิท
- **Local database ในเครื่อง:** ใช้ IndexedDB (ผ่าน Dexie.js หรือ RxDB) เป็นแหล่งข้อมูลหลักที่แอปอ่าน/เขียนตลอดเวลา — ห้ามเรียก backend ตรงๆ ตอนขายของ
- **Backend/Cloud sync:** Firebase Firestore (มี offline persistence ในตัว ใช้ฟรีได้ในแผน Spark สำหรับ data ระดับนี้) — sync ขึ้นคลาวด์เมื่อมีเน็ตเท่านั้น ไม่ block การใช้งานหลัก
- Auth: Firebase Authentication ด้วย Google Sign-In

3. Data Model (ออกแบบรองรับหลายศิลปินตั้งแต่ day 1 แม้ v1.0 จะเปิดใช้แค่คนเดียว)

```
Shop
 - id, name, owner_user_id, promptpay_id, promptpay_name

Artist
 - id, shop_id, name
 (v1.0: สร้าง default artist 1 คน = เจ้าของร้าน อัตโนมัติ ไม่ต้องมี UI จัดการหลายคนก็ได้)

Category
 - id, shop_id, name

Product
 - id, shop_id, artist_id, name, image, price, cost, category_id, stock,
is_bestseller

Event (งาน/อีเวนต์)
 - id, shop_id, name, start_date, end_date, is_active

Order
 - id (client-generated UUID), shop_id, event_id, status (completed/voided),
   payment_method (cash/qr), total, cash_received, change, created_at,
   synced (bool), created_offline (bool)

OrderItem
 - id, order_id, product_id, artist_id (snapshot ตอนขาย ห้าม join ทีหลัง),
   name_snapshot, price_snapshot, qty, discount_amount

StockMovement
 - id, product_id, order_id (nullable), type (sale/void/manual_adjust/import),
   qty_change, created_at
 (ห้ามเขียนทับ stock ตรงๆ — ทุกการเปลี่ยนแปลงสต็อกต้องผ่านตารางนี้ เพื่อกัน conflict ตอน sync)
```

กติกาสำคัญ: ทุก record ที่สร้างฝั่ง client ต้องมี `id` เป็น UUID สร้างเองตั้งแต่ต้น (ไม่รอ backend generate) เพื่อให้ sync ชนกันได้น้อยที่สุดเวลาหลายเครื่อง/offline นาน

4. ฟีเจอร์ที่ต้องมีก่อน v1.0 (เรียงตามลำดับที่ควรสร้าง)

4.1 Auth & Setup

- [ ] ล็อกอินด้วย Google
- [ ] สร้างร้านครั้งแรก (shop name, PromptPay ID)

4.2 คลังสินค้า (Inventory)

- [ ] เพิ่ม/แก้ไข/ลบสินค้า (ชื่อ, รูป, ราคา, ราคาทุน, หมวดหมู่, สต็อก)
- [ ] คัดลอกสินค้า (duplicate)

- [ ] Import สินค้าจำนวนมากจาก CSV พร้อมหน้า preview ก่อนยืนยัน (เพิ่มใหม่ vs อัปเดต, ตั้งค่าสต็อกใหม่ vs บวกเพิ่ม)
- [ ] Export รายการสินค้าเป็น CSV/Excel

4.3 หน้าขาย (Point of Sale) — หน้าหลักของแอป

- [ ] ตารางสินค้าแบบการ์ดรูปใหญ่ กดเพิ่มลงตะกร้า เหมาะกับนิ้วสัมผัส iPad
- [ ] ค้นหาสินค้าด้วยชื่อ + กรองตามหมวดหมู่
- [ ] แสดงจำนวนสต็อกคงเหลือบนการ์ดแบบเรียลไทม์ (อัปเดตจาก local DB ทันที)
- [ ] ตะกร้า: ปรับจำนวน, ลบรายการ, ใส่ส่วนลด (% หรือจำนวนเงิน ทั้งรายการ/ทั้งบิล)
- [ ] ชำระเงินสด: ใส่จำนวนที่รับมา + ปุ่มลัด (พอดี / +20 / +50 / +100 / +500 / +1000) + คำนวณเงินทอนอัตโนมัติ
- [ ] ชำระ QR พร้อมเพย์: สร้าง QR ระบุยอดจากฝั่ง client (ไลบรารี promptpay-qr หรือเทียบเท่า — ทำงานได้โดยไม่ต้องใช้เน็ต) + ปุ่มยืนยันด้วยมือ (ไม่เช็กยอดอัตโนมัติ)
- [ ] ตัดสต็อกอัตโนมัติทันทีที่ปิดการขาย (ผ่าน StockMovement)
- [ ] หน้า “Checkout Success”

4.4 ประวัติการขาย & Void

- [ ] ดูประวัติการขายทั้งหมด เรียงตามเวลาล่าสุด
- [ ] Void รายการขาย (คืนสต็อกอัตโนมัติ, ทำได้แม้ offline, ยืนยันก่อนทำเพราะย้อนกลับไม่ได้)

4.5 รายงาน (Reports)

- [ ] การ์ดสรุป: ยอดขายวันนี้ / ยอดรวมทั้งหมด / จำนวนบิล
- [ ] กราฟสินค้าขายดี Top 5
- [ ] กราฟยอดขายตามช่วงเวลาในวัน
- [ ] กรองตามอีเวนต์และช่วงวันที่
- [ ] Export รายงานเป็น Excel

4.6 อีเวนต์ (Event)

- [ ] สร้าง/แก้ไข/ลบ อีเวนต์ (ชื่องาน, วันที่)
- [ ] เลือก "อีเวนต์ที่กำลังใช้งาน" เพื่อกรองหน้าขาย
- [ ] รายงานกรองแยกตามอีเวนต์ได้

4.7 Offline Sync Engine (ฝังอยู่ในทุกฟีเจอร์ด้านบน ไม่ใช่ฟีเจอร์แยก)

- [ ] เขียนทุกอย่างลง local DB (IndexedDB) ก่อนเสมอ ไม่รอ network response
- [ ] แถบ/ไอคอนสถานะแสดงจำนวนรายการที่ยัง "รอ sync" ให้เห็นตลอดเวลา
- [ ] Sync ขึ้น Firestore อัตโนมัติเมื่อกลับมามีเน็ต (background, ไม่ block UI)
- [ ] **บล็อกการ logout ถ้ายังมีรายการค้าง sync** (บทเรียนจาก Loyverse ที่ปล่อยให้ logout แล้วข้อมูลหาย)
- [ ] Cache รูปสินค้าทั้งหมดล่วงหน้าตอนออนไลน์ (กันปัญหารูปไม่ขึ้นตอน offline แบบที่ MakerGO เจอ)

5. สิ่งที่ตั้งใจ "ไม่ทำ" ใน v1.0 (ตัดออกชัดเจน กันขอบเขตบวม)

- ระบบหลายศิลปินแชร์บูธแบบเต็มรูปแบบ (multi-artist cart, commission, settlement) — เก็บแค่ field `artist_id` ไว้ในโครงสร้างข้อมูล แต่ยังไม่ทำ UI

- เชิญทีมงาน/หลายผู้ใช้ (multi-user invite)
- Variant สินค้า (ขนาด/ลาย), bundle/โปรโมชัน
- สแกนบาร์โค้ด, พิมพ์ใบเสร็จ Bluetooth
- บัตรเครดิต/payment gateway
- Partial void (void ได้แค่ทั้งบิล ยังไม่ต้องแยกรายชิ้น)

6. Non-functional requirements

- ใช้งานได้ลื่นบน Safari/iPad แม้ปิด Wi-Fi สนิท
- ข้อมูลการเงิน (Order, StockMovement) ต้องไม่สูญหายแม้ปิดแอปกะทันหันก่อน sync
- UI ต้องออกแบบให้กดด้วยนิ้วบนจอสัมผัส ปุ่มขนาดใหญ่พอ ไม่ใช่ UI สำหรับเมาส์

ถ้าจะให้ครบกว่านี้ ผมแนะนำเพิ่ม acceptance criteria สั้นๆ ต่อฟีเจอร์ (เช่น "ปิดเน็ตแล้วขายได้ 5 บิล เปิดเน็ตใหม่ต้อง sync ขึ้นครบทั้ง 5") ให้ Antigravity ใช้เช็คงานตัวเองได้ ถ้าต้องการให้ผมช่วยร่างส่วนนั้นเพิ่มบอกได้ครับ หรือจะให้ผมใส่ brief นี้ต่อท้ายใน Claude Doc ที่เปิดอยู่ด้วยก็ได้

create file name plan.md and put all Dev Brief — Booth POS App v1.0 (สำหรับส่งต่อให้ Antigravity IDE) into

Created a file, shared files ▸
สร้างไฟล์ `plan.md` เรียบร้อยแล้วครับ เอาไปวางให้ Antigravity IDE ต่อได้เลย

ช่วยเขียนสรุปออกแบบระบบให้รองรับกับการทำงานหลายศิลปินให้ antigravity ide ทำต่อ เขียนต่อจากสรุป plan.md ล่าสุด แล้วอัปเป็น plan_v1.5.md
เหมือน v1.5 ที่สรุปกันมา

- ระบบหลายศิลปินแชร์บูธแบบเต็มรูปแบบ (multi-artist cart, commission, settlement) — เก็บแค่ field `artist_id` ไว้ในโครงสร้างข้อมูล แต่ยังไม่ทำ UI
- เชิญทีมงาน/หลายผู้ใช้ (multi-user invite)

Created a file, shared files ▸

Continue

Picking up work on the previously drafted plan.
สร้าง `plan_v1.5.md` เสร็จแล้วครับ ต่อยอดจาก `plan.md` ครบทั้ง 9 หัวข้อ เน้น multi-artist cart, commission/settlement, และระบบเชิญทีมงานตามที่สรุปไว้

ตอนนี้มีไฟล์ให้ Antigravity IDE แล้ว 2 ตัว:

- **plan.md** — v1.0 MVP (offline-first, ขายคนเดียว, โครงสร้างข้อมูลเผื่อหลายศิลปิน)
- **plan_v1.5.md** — v1.5 (เปิดใช้งานหลายศิลปิน + เชิญทีมงาน)

จะให้ผมช่วยต่อเรื่องไหนดีครับ เช่น:

- ร่าง SQL schema **จริง** (CREATE TABLE พร้อม foreign key) จากทั้งสองไฟล์นี้
- เขียน **acceptance criteria** ต่อฟีเจอร์ให้ Antigravity เช็คงานตัวเองได้
- ร่าง flow หน้าจอ (wireframe คร่าวๆ) ของตะกร้า multi-artist หรือหน้า Settlement
- สรุป **v2** ต่อ (partial void, role “artist”, multi-device sync ในบูธเดียว)

เพิ่ม feature เหมือนการเพิ่ม folder แยกของแต่ละประเภท เพราะติดปัญหาบางแอพทำมาให้ดูของได้ยาก และขียน acceptance criteria ต่อฟีเจอร์ให้ Antigravity เช็คงานตัวเองได้ มาก่อน

Read a file, ran a command, shared files
อัปเดต `plan.md` เสร็จแล้วครับ เพิ่ม 2 ส่วนหลัก:

1. หมวดหมู่แบบโฟลเดอร์ (ข้อ 4.2b) — แก้ปัญหาแท็บหมวดหมู่แนวนอนที่หาของยากเมื่อสินค้าเยอะ

- `Category` เพิ่ม `parent_category_id` ทำโฟลเดอร์ซ้อนได้ 2 ชั้น (หลัก → ย่อย)
- เข้าหน้าแรกเจอการ์ดโฟลเดอร์ แตะเข้าไปดูของข้างใน มี breadcrumb ย้อนกลับ
- มีโหมด "ดูทั้งหมด" แบบ flat list ไว้เสมอ และค้นหาข้ามโฟลเดอร์ได้ทันทีโดยไม่ต้องไล่เข้าไป
- หน้าตั้งค่าจัดการโฟลเดอร์ (สร้าง/แก้/ลบ/ลากจัดลำดับ) พร้อมกติกาเวลาลบโฟลเดอร์ที่มีของอยู่ข้างใน

2. Acceptance Criteria ทุกกลุ่มฟีเจอร์ (4.1–4.7) — เขียนเป็นเงื่อนไขทดสอบได้จริง ไม่ใช่คำอธิบายกว้างๆ เช่น

- "ปิด Wi-Fi แล้วขายครบ 5 บิล → ทุกบิลต้องขึ้นสถานะรอ sync และสต็อกต้องถูกตัดถูกต้องครบทันทีโดยไม่ต้องรอเน็ต"
- “double tap ปุ่มยืนยันชำระเงิน → ต้องสร้างออเดอร์แค่ 1 รายการ ห้ามตัดสต็อกซ้ำ”

Antigravity เอาไปเช็คงานตัวเองทีละกลุ่มได้เลย ผ่านครบทุกข้อค่อยปิดงานกลุ่มนั้น

ถ้าต้องการ ผมช่วยเขียน acceptance criteria ให้ `plan_v1.5.md` ต่อแบบเดียวกันได้ครับ

Plan.md อันนี้กับ Plan v1.5 ที่เขียนมาต่างกันมั้ย ถ้าไม่ต่างรวมให้เป็นไฟล์เดียวแล้วเขียน Planv1.5.md ให้ antigravity ide ไปทำต่อ

Consolidating both plans into one self-contained file

ต่างกันครับ ไม่ใช่ไฟล์ซ้ำ — `plan.md` คือ scope ของ v1.0 (ขายคนเดียว, โครงสร้างข้อมูลเผื่อหลายศิลปินแต่ยังไม่เปิด UI) ส่วน `plan_v1.5.md` คือส่วนที่**ต่อยอด**เพิ่มระบบหลายศิลปิน/หลายผู้ใช้เข้าไป ไม่ได้เขียนทับกัน

แต่เข้าใจว่าอยากได้ไฟล์เดียวจบให้ Antigravity ไม่ต้องสลับอ่าน 2 ไฟล์ ผมเลยรวมเป็นไฟล์เดียวให้ พร้อมเพิ่ม acceptance criteria ให้ฟีเจอร์ v1.5 ด้วย (ค้างไว้จากรอบที่แล้ว)

Ran a command, shared files

รวมเสร็จแล้วครับ `plan_v1.5.md` ตอนนี้เป็นไฟล์เดียวจบ ไม่ต้องอ่าน `plan.md` ประกอบอีก

สรุปสิ่งที่รวมเข้าไป:

- **Data model เต็ม:** เพิ่ม `Member`, `Invite`, `Settlement`, `EventArtist` เข้าไปในโครงสร้างเดิม พร้อม `commission_rate` ใน Artist
- **ฟีเจอร์ 12 กลุ่ม** (4.1–4.12) เรียงลำดับการสร้างใหม่ — แทรกฟีเจอร์หลายศิลปิน/หลายผู้ใช้ (4.5–4.8) ไว้ตรงกลางตามจุดที่ควรเริ่มทำหลังมีหน้าขายพื้นฐานแล้ว
- **Acceptance criteria ครบทุกกลุ่ม** รวมของเดิม (v1.0) และของใหม่ (multi-artist cart, commission/settlement, team invite, visibility) เช่น "ใส่ส่วนลดทั้งบิล 100 บาท กับบิลที่มีของ A มูลค่า 300 และ B มูลค่า 700 → ส่วนลดต้องแบ่งเป็น 30:70 ตามสัดส่วน"
- **ข้อ 5 (สิ่งที่ไม่ทำ)** ตัดรายการที่ทำแล้วออก (multi-artist, multi-user) เหลือเฉพาะที่ยังเลื่อนไป v2 จริงๆ

ไฟล์ `plan.md` เดิมยังอยู่ในระบบเผื่ออยากเทียบ ถ้าไม่ใช้แล้วบอกได้ ผมลบให้ได้ครับ

ถ้าหากนำไปให้ antigravity ide ทำแล้วอยากต้องปรับแก้ต้องการไฟล์ตัวไหนเพื่อที่จะดูและช่วยตัดสินใจ

ต้องการ 3 **ประเภทไฟล์** ขึ้นอยู่กับว่าจะแก้อะไรครับ

---

1. ไฟล์ที่ต้องการเสมอ (ทุกกรณี)

`plan_v1.5.md` — ไฟล์ที่เราทำไว้แล้ว ใช้เป็น source of truth เทียบกับสิ่งที่ Antigravity สร้างออกมาว่าตรงกับที่วางแผนไหม

---

2. ไฟล์จาก Antigravity ที่ต้องขอมาด้วย

แบ่งตามประเภทการแก้:

ถ้าอยากแก้ Logic / พฤติกรรมของแอป

- `schema` หรือไฟล์ data model ที่ Antigravity สร้าง (เช่น Dexie schema, Firestore rules)
- ไฟล์ที่เกี่ยวกับฟีเจอร์นั้นโดยตรง เช่น ถ้าคำนวณส่วนลดผิด ขอไฟล์ `cart.ts` หรือ `order.ts`

ถ้าอยากแก้ UI / หน้าตา

- Screenshot หรือ screen recording หน้าที่มีปัญหา
- ไฟล์ component นั้นๆ เช่น `POSScreen.tsx`, `CategoryFolder.tsx`

ถ้าอยากแก้ Offline sync / ข้อมูลหาย

- ไฟล์ sync engine เช่น `syncManager.ts`, `db.ts`
- Log หรือ error message จาก console

ถ้าอยากแก้ รายงาน / ตัวเลขผิด

- ไฟล์ที่คำนวณรายงาน เช่น `reports.ts`, `settlement.ts`
- ข้อมูลตัวอย่างที่คำนวณแล้วผิด (บอกว่าควรได้เท่าไหร่ ได้จริงเท่าไหร่)

---

3. ไฟล์เสริมที่มีประโยชน์มาก (ถ้า Antigravity สร้างไว้)

- `README.md` ของโปรเจกต์ — ดูว่า Antigravity เข้าใจ scope ถูกไหม
- **โครงสร้างโฟลเดอร์** (`tree` หรือ directory listing) — ดูว่าแบ่ง component/module ออกมาสมเหตุสมผลไหม
- Test file ถ้ามี — เช็คว่า acceptance criteria ที่เราเขียนไว้ถูก implement เป็น test จริงไหม

วิธีที่ง่ายที่สุดในทางปฏิบัติ

ให้บอก Antigravity ว่า **"ส่งออกไฟล์ที่เกี่ยวข้องกับฟีเจอร์ X มาให้"** แล้วเอาไฟล์นั้นมาให้ผมดูพร้อมกับ `plan_v1.5.md` ผมจะเปรียบเทียบให้ได้เลยว่า

- สิ่งที่สร้างออกมา **ตรงกับที่วางแผนไหม**
- Acceptance criteria ข้อไหนที่ยังไม่ผ่าน
- ควร **แก้ที่ไหน** และ**ปรับยังไง**

ถ้าอยากให้ผมช่วยเขียน prompt สำหรับส่งให้ Antigravity เริ่มงานได้เลยก็บอกได้ครับ

> https://github.com/Pitchaya-Waree/Mypos อันนี้คือข้อมูล github ของ MyPOS ถ้าจะเอาข้อมูลเพิ่มเติมให้อ่านจาก github อยากให้เขียน planv1.6.md ให้แยกละเอียดเฉพาะว่า ของๆร้าน A หรือร้าน B ขายสินค้าชิ้นไหนของตัวเองไปบ้าง

Read a page, created a file, shared files ▸

สร้าง `plan_v1.6.md` แล้วครับ แต่มีข้อจำกัดที่ต้องบอกก่อน: **repo ไม่มีข้อมูลสินค้าหรือยอดขายของร้าน A และร้าน B** ตารางในหัวข้อ 4 จึงยังว่างอยู่ ผมไม่ได้เดาตัวเลขขึ้นมาเอง

สิ่งที่พบจาก GitHub:

- **README ตรงกับแผน v1.5 เกือบทั้งหมด** ทั้ง offline-first, หมวดหมู่โฟลเดอร์, ตะกร้าหลายศิลปิน, ส่วนลดตามสัดส่วน, Settlement, PromptPay EMVCo และการซ่อนราคาทุน
- **เทคโนโลยีต่างจากแผน:** repo ใช้ Flutter + Hive ส่วนแผนเดิมเป็น PWA + Dexie ต้องตัดสินใจว่าจะไปทางไหน
- **โค้ดแอปอาจไม่ครบใน repo:** README อ้างโฟลเดอร์ `booth_pos_app/` แต่ไม่เห็นในรายการไฟล์ระดับ root

ข้อมูลที่ต้องการเพื่อเติมตารางร้าน A/B:

1. Export CSV/Excel จากหน้า Reports ของแต่ละร้าน
2. หรือ screenshot หน้า Reports/Settlement
3. หรือยืนยันว่าร้าน A และ B เป็นข้อมูลทดสอบ ผมจะสร้าง seed data ให้แทนได้

ส่งข้อมูลมาได้เลยครับ แล้วผมจะเติมตารางในไฟล์ให้ พร้อมตอบคำถามเรื่องเลือก Flutter หรือ PWA ด้วย

> จากที่แชทที่คุยกันมาอ่าน link github ก่อนจากนั้นขั้นตอนต่อไปคือวางแพลนทำ data จะเริ่มจาก firebase ซึ่งฉันไม่เคยใช้มาก่อนช่วยบอกขั้นตอนอย่างละเอียดหรือมี ai tool ตัวไหนที่สามารถเชื่อม firebase ได้เหมือนอย่าง antigravity ide ก้บอกด้วย และเริ่มทำการออกแบบ database firebase ให้เข้ากับระบบทุกอย่างอย่างเสถียรมีคำถามหรือความต้องการข้อมูลอะไรให้ถามก่อนหรือขอเสนอมาก่อน

ทดสอบการตั้งค่าด้วยเอกสารตัวอย่างและกฎความปลอดภัย ▸

ทำได้ครับ และผมแนะนำให้ทำขั้นตอน Firebase พื้นฐานก่อนได้เลย เพราะไม่ขึ้นกับการตัดสินใจข้อไหน ส่วนการออกแบบ database ผมเสนอร่างไว้ด้านล่าง พร้อมคำถามที่ต้องตอบก่อนเริ่มสร้างจริง

ส่วนที่ 1: ตั้งค่า Firebase ทีละขั้น (ทำได้เลย)

ขั้นที่ 1: สร้างโปรเจกต์

1. เข้า https://console.firebase.google.com แล้วล็อกอินด้วย Google
2. กด **Create a project** ตั้งชื่อ เช่น `booth-pos-dev`
3. ขั้น Google Analytics ปิดไว้ก่อนได้ ไม่จำเป็นสำหรับตอนนี้

ขั้นที่ 2: เปิดฐานข้อมูล Firestore

1. เมนูซ้าย **Build → Firestore Database → Create database**
2. เลือก Location เป็น **asia-southeast1 (Singapore)** เพื่อให้ใกล้ไทยที่สุด (เลือกแล้วเปลี่ยนไม่ได้)
3. เลือก **Production mode** ระบบจะปิดการเข้าถึงทั้งหมดไว้ก่อน แล้วเราค่อยเขียนกฎเอง
   - อย่าใช้ Test mode ระยะยาว เพราะมันจะเปิดให้ทุกคนอ่าน/เขียนได้ และหมดอายุใน 30 วัน

ขั้นที่ 3: เปิดการล็อกอินด้วย Google

1. Build → Authentication → Get started
2. แท็บ Sign-in method → เลือก Google → Enable → ใส่อีเมลสนับสนุน แล้ว Save

ขั้นที่ 4: ลงทะเบียนแอป เพื่อรับค่า config

1. ไอคอนเฟือง Project settings → General → Your apps
2. กดไอคอน Web (`</>`) ตั้งชื่อแอป แล้วคัดลอกค่า `firebaseConfig` เก็บไว้
   - ค่าพวกนี้ไม่ใช่รหัสลับ ความปลอดภัยอยู่ที่ Security Rules ไม่ได้อยู่ที่การซ่อนค่า config

ขั้นที่ 5: ติดตั้งเครื่องมือบนคอมพิวเตอร์

1. ติดตั้ง Node.js (LTS) จาก nodejs.org
2. เปิดเทอร์มินัลแล้วรัน

```
npm install -g firebase-tools
firebase login
```

3. ถ้าใช้ Flutter (ตาม repo) ให้รันเพิ่ม

```
dart pub global activate flutterfire_cli
flutterfire configure
```

คำสั่งนี้จะเชื่อมโปรเจกต์ Flutter กับ Firebase ให้อัตโนมัติ

ขั้นที่ 6: เขียน Security Rules เบื้องต้น

สร้างไฟล์ `firestore.rules` แล้วใส่โค้ดตัวอย่างนี้ (เป็นโครงเบื้องต้น ยังไม่พอสำหรับใช้จริง จะปรับตามการตัดสินใจด้านล่าง)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function isMember(shopId) {
      return request.auth != null &&

exists(/databases/$(db)/documents/shops/$(shopId)/members/$(request.auth.uid));
    }
    match /shops/{shopId}/{document=**} {
      allow read, write: if isMember(shopId);
    }
  }
}
```

deploy ด้วย `firebase deploy --only firestore:rules`

ขั้นที่ 7: Blaze plan (ยังไม่ต้องอัปเกรดตอนนี้)

- ต้องอัปเกรดเมื่อจะใช้ Storage (เก็บรูปสินค้า) หรือ Cloud Functions (เช่น ระบบเชิญทีม)
- ก่อนอัปเกรดให้ตั้ง **Budget alert** ที่ Google Cloud Billing ทุกครั้ง

ส่วนที่ 2: เครื่องมือ AI ที่เชื่อม Firebase ได้

- **Firebase MCP server:** Firebase มี MCP server ที่ให้ AI อ่านโปรเจกต์ สร้าง/แก้ rules และ query ข้อมูลได้โดยตรง ใช้ได้กับ MCP client เช่น Claude Code
- **Claude Code:** รันคำสั่ง `firebase` และ `flutterfire` ในเทอร์มินัลได้เอง จัดการ deploy rules และเขียนโค้ดเชื่อม Firestore ได้ครบ
- **Firebase Studio:** IDE บนคลาวด์ของ Google ที่มี Firebase และ Gemini ติดมาให้

ผมไม่แน่ใจว่า **Antigravity IDE รองรับ Firebase MCP โดยตรงแค่ไหน** แนะนำให้เช็กเอกสารของมันก่อน ถ้าไม่รองรับ ให้ใช้ Antigravity เขียนโค้ดแอป แล้วใช้ Claude Code หรือ Firebase CLI จัดการด้าน backend และ rules

---

ส่วนที่ 3: ร่างการออกแบบ Firestore (รอยืนยันก่อนสร้างจริง)

ใช้โครงแบบ **แยกข้อมูลตามร้านด้วย subcollection ใต้** `shops/{shopId}` เพราะ Security Rules เขียนง่าย และข้อมูลร้าน A กับร้าน B แยกกันชัดเจนตั้งแต่โครงสร้าง

```
shops/{shopId}
  name, promptpayId, promptpayName, ownerUid, createdAt

  members/{uid}              ← doc id = uid ทำให้ rules เช็กสิทธิ์ได้เร็ว
    role (owner/member), artistId, joinedAt

  invites/{token}
    email, expiresAt, status (pending/accepted/expired/revoked)

  artists/{artistId}
    name, commissionRate, linkedUid

  categories/{categoryId}
    name, parentId (null = root), sortOrder

  products/{productId}       ← ข้อมูลที่ทุกคนในร้านเห็นได้
    artistId, categoryId, name, imageUrl, price, stock, isBestseller, updatedAt

  productCosts/{productId}   ← แยกต้นทุนออกมา (ดูข้อ 1 ด้านล่าง)
    cost, ownerArtistUid

  events/{eventId}
    name, startDate, endDate, isActive
    artists/{artistId}       ← EventArtist (ศิลปินที่มาออกงานนี้)
    hiddenProducts/{productId}

  orders/{orderId}           ← orderId = UUID ที่ client สร้าง
    eventId, status (completed/voided), paymentMethod, total,
    cashReceived, change, createdBy, createdAt
    items/{itemId}
      productId, artistId, nameSnapshot, priceSnapshot, qty, discount

  stockMovements/{movementId}
    productId, orderId, type (sale/void/manual_adjust/import), qtyChange,
createdAt

  settlements/{eventId_artistId}
    grossSales, commissionAmount, netPayout, paid, paidAt, note
```

จุดที่ผมเสนอให้เปลี่ยนจากแผนเดิม และเหตุผล

1. **แยก `productCosts` ออกจาก `products`** : Firestore Rules ทำงานระดับ document ไม่สามารถซ่อนบาง field ได้ ถ้าเก็บ cost ไว้ใน products ทุกคนที่อ่านได้จะเห็นต้นทุนทุกชิ้น การแยกเอกสารออกมาทำให้กฎ "Member เห็นต้นทุนเฉพาะของตัวเอง" ทำได้จริง
2. **สต็อกใช้ ledger + ค่าที่คำนวณไว้:** `stockMovements` เป็นประวัติ ส่วน `products.stock` เป็นยอดปัจจุบันที่อัปเดตด้วย `FieldValue.increment` ในทรานแซกชันเดียวกัน ไม่เขียนทับค่าตรงๆ ตาม

กติกาเดิม

3. **รายงานใช้ aggregation query** (sum/count) ของ Firestore ได้ ไม่ต้องดึงทุกออเดอร์มาคำนวณเอง

ส่วนที่ 4: คำถามที่ต้องตอบก่อนสร้างจริง

1. **Flutter หรือ PWA?** repo ใช้ Flutter แต่แผนเดิมเป็น PWA ผมแนะนำ Flutter ถ้าจะยึด repo นี้เป็นฐาน เพราะโค้ดที่มีอยู่ใช้ได้เลย
2. **ใช้ Firestore offline cache อย่างเดียว หรือคง Hive ไว้ด้วย?** Firestore มีระบบ cache ในตัวอยู่แล้ว ถ้าใช้ Flutter ควรเลือกอย่างใดอย่างหนึ่งเพื่อกันข้อมูลซ้ำซ้อน ผมแนะนำให้ใช้ Firestore cache อย่างเดียว
3. **สต็อกติดลบได้ไหม?** ถ้า 2 เครื่องขายชิ้นสุดท้ายตอน offline แล้วซิงก์ทีหลัง ผมแนะนำให้ยอมให้ติดลบแล้วแจ้งเตือนเจ้าของให้ปรับสต็อก
4. **รูปสินค้า:** จะอัปเกรด Blaze เพื่อใช้ Storage หรือจะย่อรูปเก็บไว้ใน Firestore (จำกัดขนาด 1 MB ต่อเอกสาร) ผมแนะนำ Storage เพราะเสถียรกว่า
5. **เจ้าของหลายร้าน:** ถ้า Google account เดียวต้องดูแลทั้งร้าน A และร้าน B ได้ จะต้องปรับกติกา "1 คน 1 ร้าน" ในแผนเดิม ต้องการแบบไหน

ตอบข้อ 1–2 มาก่อนได้เลย แล้วผมจะปรับ Security Rules และร่างโครงสร้างให้ละเอียดขึ้นพร้อม acceptance criteria สำหรับ Firebase โดยเฉพาะครับ

1.ใช้ flutter
2.firestore cache
3.สต็อกติดลบได้
5.google accounts เดียวสามารถดูแลได้มากกว่า 1 ร้านซึ่ง 1 ร้านมี 1 เจ้าของ
อยากรู้ว่ามีค่าใช้จ่ายอะไรมั้ยเพราะอยากให้ระบบออกมาเล็กๆก่อนโดยยังมี user ไม่เยอะ และอธิบายคำถามที่ 4 เพิ่มเติม

Read and updated memory ▸

ได้ข้อสรุปแล้วครับ ขอสรุปการตัดสินใจก่อน แล้วอธิบายเรื่องค่าใช้จ่ายและคำถามข้อ 4

การตัดสินใจที่บันทึกแล้ว

หัวข้อ	ตัดสินใจ
แพลตฟอร์ม	Flutter ตาม repo Mypos
Local data	ใช้ Firestore offline cache อย่างเดียว ไม่ใช้ Hive
สต็อก	ติดลบได้ แล้วแจ้งเตือนเจ้าของให้ปรับ
บัญชี Google	1 บัญชีดูแลได้หลายร้าน แต่ 1 ร้านมี 1 เจ้าของ

ข้อ 4 ยังไม่ได้ตัดสินใจ ผมอธิบายด้านล่างแล้วให้เลือกได้เลย

ค่าใช้จ่ายของ Firebase

ระดับ Spark (ฟรี) ใช้ได้ถึงไหน

จากที่ตรวจไว้ก่อนหน้า Firestore ฟรีถึง 1 GiB, อ่าน 50,000 ครั้งต่อวัน, เขียน 20,000 ครั้งต่อวัน และ Authentication ฟรีถึง 50,000 MAU

ระดับ Blaze (เสียเงินตามการใช้งาน)

Blaze เป็นแผนที่ผูกบัตรเครดิต แต่ยังมีโควตาฟรีเหมือน Spark ก่อนคิดเงินส่วนที่เกิน ต้องใช้แผนนี้ในสองกรณี

- ใช้ Firebase Storage เก็บรูปสินค้า
- ใช้ Cloud Functions ทำระบบเชิญทีม (ต้องตรวจ token ฝั่งเซิร์ฟเวอร์เพื่อความปลอดภัย)

ประเมินสำหรับร้านเล็ก

สมมติร้านหนึ่งมี 200 ออเดอร์ต่อวัน แต่ละออเดอร์ 5 รายการ จะมีการเขียนประมาณ 16 ครั้งต่อออเดอร์ รวมประมาณ 3,200 ครั้งต่อวัน ซึ่งยังต่ำกว่าโควตาฟรี 20,000 ครั้ง ดังนั้น**ถ้าใช้งานเล็กๆ ค่าใช้จ่ายรายเดือนน่าจะใกล้ 0 บาท**

อย่างไรก็ตาม ตัวเลขราคาต่อหน่วยของ Blaze ผมไม่ได้ยืนยันจากหน้าราคาปัจจุบันในตอนนี้ ควรเปิดดูที่ firebase.google.com/pricing ก่อนตัดสินใจ

ข้อควรระวังเรื่องค่าใช้จ่าย

- **Budget alert ไม่ได้หยุดการใช้จ่ายให้อัตโนมัติ** มันแค่แจ้งเตือน จึงควรตั้งงบต่ำ เช่น 100–200 บาท และตั้ง quota limit ใน Google Cloud Console เพิ่มเติมเพื่อจำกัดการเขียนหรืออ่านรายวัน
- **อย่าอ่านทั้ง collection ทุกครั้ง** เพราะคิดค่าอ่านต่อเอกสาร ควรใช้ query ที่มีเงื่อนไขและ limit
- Offline cache ช่วยลดการอ่านซ้ำ ซึ่งเป็นผลดีต่อค่าใช้จ่าย

อธิบายคำถามข้อ 4: จะเก็บรูปสินค้าอย่างไร

มี 2 ทางเลือก

ทางที่ 1: Firebase Storage (แนะนำ)

- เก็บไฟล์รูปแยกต่างหาก แล้วใน Firestore เก็บแค่ลิงก์รูป
- ย่อรูปก่อนอัปโหลดให้เหลือขนาดประมาณ 300 พิกเซลด้าน และไม่เกิน 150 KB ต่อรูป
- สินค้า 500 ชิ้นจะใช้พื้นที่ประมาณ 75 MB ซึ่งอยู่ในโควตาฟรีของ Storage ที่ประมาณ 5 GB
- ต้องใช้แผน Blaze
- ข้อดี: รูปไม่ทำให้เอกสาร Firestore บวม และจัดการรูปได้สะดวก
- ข้อเสีย: ต้องอัปเกรดแผนและตั้งค่า rules ของ Storage เพิ่ม

ทางที่ 2: เก็บรูปขนาดเล็กเป็นข้อความ (base64) ใน Firestore

- แปลงรูปเป็นข้อความแล้วเก็บในเอกสารสินค้าเลย
- ข้อดี: ไม่ต้องใช้ Blaze และไม่ต้องจัดการ Storage
- ข้อจำกัด: เอกสาร Firestore ใหญ่สุดได้ 1 MB ดังนั้นรูปต้องเล็กมาก (ราว 30–50 KB) รูปคุณภาพต่ำลง และทำให้เอกสารสินค้าใหญ่ ซึ่งทำให้การโหลดรายการสินค้าช้าลง

ผมแนะนำทางที่ 1 เพราะโครงการจะโตขึ้นเรื่อยๆ และค่าใช้จ่ายจากการใช้งานเล็กๆ ก็ยังน่าจะเป็นศูนย์ บวกกับการตั้ง budget alert ไว้แล้ว ความเสี่ยงต่ำมาก

สิ่งที่ต้องยืนยันก่อนเริ่มสร้าง

1. **เลือกทางเลือกรูป:** Storage (แนะนำ) หรือ base64 ใน Firestore
2. **ยืนยันการเปลี่ยนแปลงกติกา "1 คน 1 ร้าน":** เนื่องจากคุณต้องการให้ 1 บัญชีดูแลหลายร้านได้ ระบบจะต้องมีหน้าเลือกร้าน และ `members` จะเก็บความสัมพันธ์ที่หลายร้านต่อหนึ่ง uid ได้ ซึ่งผมเตรียมไว้ให้แล้วในโครงสร้าง

ถ้าตอบสองข้อนี้ ผมจะร่าง Firestore schema ฉบับละเอียดพร้อม Security Rules ที่ตรวจสิทธิ์ทั้งเจ้าของและสมาชิก และเขียนขั้นตอนตั้งค่า Flutter + FlutterFire ให้ทำตามได้ทีละขั้นครับ

1.Storage (แนะนำ)
เนื่องจาก 1 event เราต้อง upload ชิ้นงานเข้าไปเผื่อเอาไปขาย แต่ถ้าเข้า event ที่ 2 ชิ้นงานนั้นจะต้องเพิ่มใหม่ไหม หรือว่าไม่ต้องเพิ่มจะติดมาให้เลยแล้วถ้าหากให้มีเหมือน storage ไว้เก็บงานเราคล้ายๆ folder กลางที่ยังไม่เข้ากับ event ไหนเลยพอเมื่อเจ้าของร้านสร้าง 1 event ที่จะไปออกก้สามารถดึงชิ้นงานจาก storage กลางนั้นได้เลยโดยไม่ต้อง upload ใหม่

Confirming images live in shop-level product storage ▸
ใช่ครับ โครงสร้างที่เราออกแบบไว้รองรับแบบนี้ได้เลย ไม่ต้องอัปโหลดใหม่ทุกงาน เพราะ**สินค้าและรูปผูกกับร้าน ไม่ได้ผูกกับงาน** งานแค่เลือกว่าจะเอาสินค้าชิ้นไหนออกขาย

แนวคิด: คลังกลางของร้าน + งานเป็นแค่การเลือก

```
shops/{shopId}
  products/{productId}          ← คลังกลาง: ทุกชิ้นของร้านอยู่ที่นี่ ไม่สังกัดงาน
    imageUrl → Storage: shops/{shopId}/products/{productId}.jpg   (อัปโหลดครั้งเดียว)

  events/{eventId}              ← งาน เลือกจากคลังกลาง
    artists/{artistId}          ← ศิลปินที่มาออกงานนี้
    includedProducts/{productId} ← สินค้าที่นำมาขายในงานนี้ (หรือ hiddenProducts แบบเดิม)
```

- **รูปเก็บครั้งเดียว** ใน Storage ตาม `productId` งานไหนก็อ้างถึงรูปเดิมผ่าน `imageUrl`
- **งานที่ 2 ไม่ต้องอัปโหลดใหม่** เพียงเลือกสินค้าจากคลังกลางมาใส่งานนั้น
- **ปุ่ม "คัดลอกสินค้า" (duplicate)** ที่มีอยู่แล้วจะคัดลอกแค่ข้อมูล ส่วนรูปใช้ `imageUrl` เดิมร่วมกัน จึงไม่เปลืองพื้นที่และไม่ต้องอัปโหลดซ้ำ
- **ลบสินค้าจากงาน** ทำแค่ลบออกจาก `includedProducts` ไม่กระทบคลังกลางหรือรูป
- **ลบสินค้าจากคลังกลาง** ต้องถามก่อนเสมอ เพราะรูปและสินค้าอาจยังถูกใช้อยู่ในงานอื่น หรือในประวัติการขาย (ประวัติใช้ `nameSnapshot` อยู่แล้ว จึงไม่พัง)

ขั้นตอนที่เจ้าของร้านจะเห็น (UX)

1. สร้างงานใหม่ → ตั้งชื่องาน วันที่
2. เลือกศิลปินที่มาออกงาน
3. หน้าเลือกสินค้าจาก "คลังกลาง" มีช่องค้นหาและโฟลเดอร์แบบเดียวกับหน้าคลัง ติ๊กชิ้นที่จะขาย
4. ปุ่ม **"ใช้ชุดเดียวกับงานล่าสุด"** เพื่อให้งานที่ 2 ตั้งค่าเกือบเสร็จในคลิกเดียว

คำถามก่อนลงรายละเอียด

สต็อกเป็นของกลางหรือแยกตามงาน?

- **แบบกลาง:** สินค้า 1 ชิ้นมีสต็อกเดียว งาน A ขายไป 3 ชิ้น งาน B จะเหลือเท่าเดิมตามสต็อกรวม ง่ายและตรงกับวิธีที่ผู้ขายคิด
- **แบบแยกตามงาน:** แต่ละงานมีสต็อกเริ่มต้นของตัวเอง ช่วยให้รู้ยอดขายต่องานชัด แต่ต้องเติมสต็อกให้ทุกงานเอง

ผมแนะนำ**แบบกลาง** เพราะเรื่องที่ 2 ชิ้นชิ้นเดียวกันอยู่ในงานพร้อมกันแทบไม่เกิด และไม่ต้องตั้งสต็อกซ้ำทุกงาน แต่ถ้าอยากดูยอดขายต่องานแยกชัด เราแยกได้โดยบันทึก `eventId` ในทุกออเดอร์ซึ่งเราทำอยู่แล้ว

ตอบข้อนี้มาได้เลยครับ แล้วผมจะเขียน Firestore schema ฉบับเต็มพร้อม Security Rules และ Storage rules ที่สอดคล้องกับโครงนี้

QuestionAnswer.txt
45 lines

TXT

คิดว่าดีกว่ามั้ย

Checking the uploaded Q&A against the earlier design decisions ▸

ดีกว่าครับ แต่ควรมี Owner 1 คนต่อร้าน แล้วเพิ่มบทบาทอื่นที่ไม่แตะเรื่องเงินไว้ช่วยงาน ไม่ควรเปิดให้ Owner หลายคนมีสิทธิ์เท่ากัน

ไฟล์ที่คุณอัปโหลดมาพูดถูกในหลักการ แต่มีจุดหนึ่งที่ควรแก้: สิทธิ์ Owner ผูกกับ**ร้าน (shop)** ไม่ใช่กับ Event ดังนั้นคำว่า “1 Owner ต่อ Event” จริงๆ คือ “1 Owner ต่อร้าน” ซึ่งตรงกับการตัดสินใจของเราที่ว่า **1 ร้านมี 1 เจ้าของ** ส่วนบัญชี Google หนึ่งบัญชีดูแลได้หลายร้านได้

เหตุผลที่ 1 Owner ดีกว่า

- **เงินมีทางเดียว** PromptPay, การปิดยอด Settlement และการ "จ่ายแล้ว" ต้องมีคนรับผิดชอบคนเดียว ไม่เกิดการโอนซ้ำหรือบันทึกชนกัน
- **ต้นทุนของศิลปินเป็นความลับ** Owner เห็นต้นทุนทุกชิ้น ทำให้ Member เห็นได้เฉพาะของตัวเองตามกติกาที่เราวางไว้
- **กันความผิดพลาดหน้างาน** ลดโอกาสที่คนหนึ่งเปลี่ยนเบอร์ PromptPay หรือลบข้อมูลตอนอีกคนกำลังคิดเงิน

จุดอ่อนที่ต้องแก้

Owner คนเดียวทำให้เกิด**จุดล้มเหลวจุดเดียว** ถ้า Google account ของ Owner ถูกล็อก หรือ Owner ไม่อยู่งาน ร้านทั้งร้านจะเข้าไม่ได้ ทางแก้มี 2 อย่าง:

1. **ฟังก์ชันโอนความเป็นเจ้าของ** ให้ Owner โอนสิทธิ์ให้ Manager คนที่ไว้ใจได้ ผ่านการยืนยันสองฝั่ง
2. **เก็บ backup** ข้อมูลร้านเป็นไฟล์ export ทุกสิ้นวัน เพื่อกู้ข้อมูลได้แม้บัญชีมีปัญหา

โครงสร้างบทบาทที่แนะนำ

บทบาท	ทำได้	ทำไม่ได้
Owner (1 คนต่อร้าน)	ทุกอย่าง รวม PromptPay, Settlement, ต้นทุนทุกชิ้น, โอนความเป็นเจ้าของ	-
Manager (หลายคนได้)	จัดการสินค้า หมวดหมู่ อีเวนต์ ขาย ดูรายงานยอดขาย, เชิญ Member	แก้ PromptPay, กด Settlement, เห็นต้นทุนทั้งหมด
Member	ขาย, ดูสินค้า, ดูยอดของตัวเอง	แก้สินค้า, เห็นต้นทุนที่ไม่ใช่ของตัวเอง

ส่วน Manager ตรงกับที่ไฟล์เสนอว่าเป็น "Co-Host" ซึ่งเป็นทางที่ดีที่สุดในการรองรับเพื่อนศิลปินที่ช่วยงานโดยไม่ต้องให้สิทธิ์เรื่องเงิน

ผลต่อ Firebase schema

- `members/{uid}.role` เพิ่มค่าเป็น `owner | manager | member`
- Security Rules ตรวจสิทธิ์ฝั่งเซิร์ฟเวอร์: เฉพาะ `owner` เขียน `shops.promptpayId` และ `settlements` ได้ ส่วน `manager` เขียนได้เฉพาะ products, categories, events, orders
- `productCosts` อ่านได้เฉพาะ `owner` หรือ `member` ที่ `artistId` ตรงกับเจ้าของสินค้า
- ฟังก์ชันโอนความเป็นเจ้าของต้องทำฝั่งเซิร์ฟเวอร์ (Cloud Functions) เพื่อป้องกันการโอนผิดคน

ไฟล์ที่คุณส่งมาเป็นคำตอบจากแหล่งอื่น ผมยังตรวจไม่ได้ว่าโค้ดจริงใน repo ทำงานตามที่อธิบายทุกข้อ โดยเฉพาะเรื่องการแสดงต้นทุน ดังนั้นก่อนพึ่งพาข้อมูลส่วนนี้ ควรไล่ดูว่าโค้ดตรวจสิทธิ์ตรงไหน

ถ้าเห็นด้วยกับโครงบทบาทนี้ ผมจะเขียน Firestore schema ฉบับเต็มและ Security Rules ที่รองรับ 3 บทบาทให้เลยครับ

งั้นช่วยเริ่มออกแบบจากระบบเขียนเป็น PlanV1.7.md เกี่ยวกับเรื่องที่คุยและที่ต้องแก้ จากนั้นให้เริ่มออกแบบระบบ Firebase ได้เลย

Searched the web, updated memory, and 2 more tools ▸

เขียน `plan_v1.7.md` เสร็จแล้วครับ (ผมตั้งชื่อตามชุดเดิมเป็น `plan_v1.7.md`) ในไฟล์มีการตัดสินใจที่ล็อกแล้ว รายการที่ต้องแก้จาก v1.5/v1.6 และการออกแบบ Firebase ตั้งแต่ schema, Security Rules ของ Firestore และ Storage, Cloud Functions 5 ตัว ไปจนถึง acceptance criteria และลำดับการสร้าง

จุดสำคัญที่ออกแบบไว้

- **Orders เก็บ items เป็น array ในเอกสารเดียว** และใช้ `WriteBatch` กับ `increment` ตอนขาย เพราะ transaction ทำงานตอน offline ไม่ได้ วิธีนี้ทำให้สต็อกติดลบได้โดยไม่ error และกัน void ซ้ำสองเครื่อง
- **ราคาทุนและ commission แยกเป็นเอกสารต่างหาก** เพราะ Firestore Rules ซ่อนบาง field ไม่ได้
- **คลังกลาง:** สินค้าและรูปอยู่ที่ร้าน งานเลือกสินค้าผ่าน `eventProducts` และมีปุ่ม "ใช้ชุดเดียวกับงานล่าสุด"
- **หลายร้านต่อบัญชี** ผ่าน `users/{uid}/memberships` และหน้าเลือกร้าน
- **ห้าม logout หรือสลับร้านขณะมีรายการค้างซิงก์** ไม่เช่นนั้นรายการจะถูกปฏิเสธและหาย

ค่าใช้จ่ายที่ตรวจเพิ่มแล้ว

Storage ต้องใช้ Blaze ตั้งแต่ 3 กุมภาพันธ์ 2026 แต่ยังมีโควตาฟรี ส่วน Firestore โควตาฟรียังเท่าเดิม ร้านที่ขายราว 200 ใบต่อวันน่าจะใช้ประมาณ 2,200 writes ซึ่งต่ำกว่าเพดาน 20,000 มาก ราคาเมื่อเกินโควตาแต่ละแหล่งให้ตัวเลขไม่ตรงกัน ผมจึงไม่ได้ใส่ตัวเลขราคาไว้

สิ่งที่ผมเจอระหว่างออกแบบและควรรู้

- **การส่งมอบบน iPad กระทบค่าใช้จ่าย:** แอป Flutter เนทีฟบน iOS ต้องมีบัญชี Apple Developer แบบเสียเงินรายปีสำหรับติดตั้งถาวร ส่วน Flutter Web/PWA ไม่เสียค่านี้ แต่ต้องทดสอบ offline และการล็อกอิน Google บน Safari จริง
- **ข้อมูลเรื่อง repo มาจาก README เท่านั้น** Antigravity ต้องเปิดโค้ดจริงตรวจก่อน
- **Rules ที่ผมเขียนยังไม่ได้ทดสอบ** ต้องรันผ่าน Emulator ตาม acceptance criteria ข้อ 12.1 ก่อน deploy

ต้องการให้คุณตัดสินใจ 3 เรื่อง

1. **สต็อกแบบกลาง** (ผมแนะนำ) หรือแยกตามงาน
2. **Member เห็นออเดอร์เฉพาะที่ตัวเองขาย** (ตามไฟล์) หรือเห็นทั้งร้าน
3. **ส่งมอบบน iPad แบบ PWA หรือแอปเนทีฟ**

ถ้าตอบสามข้อนี้ ผมจะปรับไฟล์ให้ตรง แล้วเขียนไฟล์ `firestore.rules`, `storage.rules`, `firestore.indexes.json` และชุด rules test แยกให้ Antigravity ใช้ต่อได้เลยครับ

1.สต็อกแบบกลาง
2.Member เห็นออเดอร์เฉพาะที่ตัวเองขาย
3. อธิบายเพิ่มเติมความต่างระหว่าง

Edited a file ▸

ขอบคุณครับ ข้อ 1 และ 2 ตรงกับที่ไฟล์ v1.7 ออกแบบไว้แล้ว ผมอัปเดตหัวข้อ 0 ให้เป็นข้อที่ยืนยันแล้ว ส่วนข้อ 3 อธิบายด้านล่าง

PWA กับแอปเนทีฟ ต่างกันอย่างไร

หัวข้อ	PWA (เว็บติดตั้งลงหน้าจอโฮม)	แอปเนทีฟ iOS (ผ่าน TestFlight)
วิธีติดตั้ง	เปิดลิงก์ใน Safari แล้วกด Share → Add to Home Screen	ติดตั้งจาก TestFlight ได้เลย
ค่าใช้จ่าย	ไม่มีค่าสโตร์	ต้องมีบัญชี Apple Developer แบบเสียเงินรายปี (ประมาณ 99 USD/ปี ตรวจราคาล่าสุดกับ Apple)
อัปเดตแอป	deploy ขึ้น Firebase Hosting แล้วเปิดใหม่ได้ทันที	อัปโหลดบิลด์ใหม่ ผ่าน TestFlight ได้
ข้อมูล offline	**เสี่ยง:** Safari อาจล้างข้อมูลใน IndexedDB ถ้าไม่ได้เปิดแอปหลายสัปดาห์	เก็บในพื้นที่แอปของระบบ ไม่ถูกล้างเอง
ฮาร์ดแวร์	เครื่องพิมพ์ Bluetooth และสแกนบาร์โค้ดผ่านกล้องทำได้จำกัด	เข้าถึงกล้อง Bluetooth และเครื่องพิมพ์ได้เต็มที่
Google Sign-In	บางครั้งเจอปัญหาหน้าต่างป๊อปอัปบน Safari	ใช้ระบบล็อกอินของ iOS ได้ตามปกติ
การแจ้งเตือน	ทำได้เฉพาะเมื่อติดตั้งลงหน้าจอโฮมแล้ว และมีข้อจำกัดบางอย่าง	ทำได้เต็มที่

จุดที่ต้องระวังมากที่สุดของ PWA บน iPad

ระบบเราเก็บข้อมูลที่ยังไม่ซิงก์ไว้ในเครื่อง ถ้า Safari ล้างข้อมูลนี้ไป **ออเดอร์ที่ค้างอยู่จะหายโดยไม่มีข้อความเตือน** ซึ่งขัดกับเป้าหมายหลักของแอป แม้จะลดความเสี่ยงได้ด้วยการเปิดแอปให้ซิงก์บ่อยๆ และแจ้งเตือนเมื่อมีรายการค้างนาน แต่ก็ควบคุมไม่ได้ทั้งหมด

ข้อสังเกตเรื่องค่าใช้จ่ายและการทำงาน

- **Flutter เขียนครั้งเดียวได้ทั้ง iPad, Android และเว็บ** ดังนั้นการเลือกช่องทางส่งมอบไม่ได้ผูกโค้ดให้ต้องเลือกตอนนี้ตลอดไป
- Apple Developer เป็นค่าใช้จ่ายรายปีเพียงรายการเดียวในระบบนี้ Firebase ยังคงเป็นค่าใช้จ่ายตามการใช้งาน
- ช่วงพัฒนา ใช้ Flutter Web ทดสอบได้เลยโดยไม่เสียค่าใช้จ่ายเพิ่ม

คำแนะนำของผม

- **ช่วงพัฒนาและทดสอบ:** ใช้ Flutter Web (PWA) ก่อน เพื่อเร็วและประหยัด
- **ใช้งานจริงหน้าบูธ:** ควรเป็น **แอปเนทีฟ iOS ผ่าน TestFlight** เพราะความน่าเชื่อถือของข้อมูลการเงินสำคัญกว่าค่าสมาชิกปีละครั้ง และเครื่องพิมพ์ใบเสร็จในอนาคตต้องใช้ Bluetooth

ตอบมาได้เลยว่าจะให้ทำแบบไหน (PWA เท่านั้น / เนทีฟ iOS สำหรับใช้จริง / ทั้งสองแบบ) แล้วผมจะเพิ่มขั้นตอนตั้งค่าที่ตรงกับเลือกนั้นในไฟล์ v1.7 และเขียน `firestore.rules`, `storage.rules` และ rules test ให้ Antigravity ใช้ต่อได้

ยังไม่เข้าใจ สิ่งที่ต้องการอยากให้บน ipad โหลด app ได้ผ่าน store จากนั้นให้ล็อกอินเข้า google acounts แล้วเข้าไปยังแอพเมื่ออยู่หน้างานจริงเปิดแอพแล้วใช้ได้เลย เหมาะกับรูปแบบไหนโดยที่ ipad ไม่ต้องเสียค่าใช้จ่าย

Edited a file, updated memory

ใช่ครับ รูปแบบที่ตรงกับที่คุณต้องการคือ **แอป iOS เนทีฟ (Flutter) เผยแพร่บน App Store** สำหรับ iPad ผู้ใช้โหลดฟรี ไม่ต้องจ่ายอะไร ส่วนค่าใช้จ่ายรายปีเป็นของผู้พัฒนา

ใครจ่ายอะไร

รายการ	ใครจ่าย	จำนวน
โหลดแอปจาก App Store	ผู้ใช้ iPad	**ฟรี**
ล็อกอิน Google	ผู้ใช้	**ฟรี**
Apple Developer Program	ผู้พัฒนา (คุณ)	ประมาณ 99 USD ต่อปี (ตรวจราคาล่าสุดกับ Apple)
Firebase (Firestore, Storage, Functions)	ผู้พัฒนา	ตามการใช้งาน ร้านเล็กน่าจะอยู่ในโควตาฟรีส่วนใหญ่

ขั้นตอนของผู้ใช้หลังติดตั้ง

1. โหลดแอปจาก App Store บน iPad
2. เปิดแอปแล้วล็อกอิน Google (ครั้งแรกต้องมีอินเทอร์เน็ต)
3. เลือกร้าน แล้วใช้งานได้ทันที
4. ครั้งถัดไปเปิดแอปหน้างานได้เลย ข้อมูลที่โหลดไว้ยังอยู่ในเครื่อง และขายได้แม้ไม่มีเน็ต

สองเรื่องที่ต้องรู้ก่อนส่งแอปขึ้น App Store

- **Sign in with Apple:** แอปที่มี Google Sign-In ต้องมี Sign in with Apple ด้วย (Guideline 4.8) ซึ่งเพิ่มเข้ามาในแผนแล้ว
- **รูปแบบการเผยแพร่:** เลือกได้ 3 แบบ
  - **Public** ทุกคนค้นหาเจอใน App Store
  - **Unlisted** ค้นหาไม่เจอ เปิดได้เฉพาะคนที่มีลิงก์ เหมาะกับกลุ่มศิลปินที่แชร์บูธกัน
  - **TestFlight** ใช้ทดสอบกับคนไม่เกิน 10,000 คน แต่บิลด์หมดอายุใน 90 วัน ต้องอัปโหลดใหม่

ผมแนะนำ **Unlisted** สำหรับใช้จริง และใช้ TestFlight ช่วงทดสอบ

ทางเลือกที่ฟรีทั้งผู้พัฒนาและผู้ใช้ แต่ไม่แนะนำ

- PWA ไม่ต้องผ่าน App Store แต่ Safari อาจล้างข้อมูล offline ทำให้ออเดอร์ค้างหายได้
- **ติดตั้งด้วย Apple ID ฟรี** แอปหมดอายุทุก 7 วัน ใช้หน้างานจริงไม่ได้

ผมอัปเดตไฟล์ `plan_v1.7.md` ในหัวข้อ 10.5 ให้ตรงกับแนวทางนี้แล้ว ตอบมาได้เลยว่าต้องการ **Unlisted** หรือ **Public** แล้วผมจะเขียนขั้นตอนสมัคร Apple Developer, ตั้ง Bundle ID และส่งขึ้น TestFlight ให้ทำตามได้ทีละขั้นครับ

planv.1.7.md ได้เขียนอธิบายให้ Antigravity ide ai ทำต่อแล้วใช่มั้ย

Checking which planned rule files are still missing ▸

ใช่ครับ `plan_v1.7.md` เขียนไว้แล้ว และ Antigravity เริ่มทำได้ แต่ **ยังไม่ครบพอให้ทำต่อจนจบ** ผมควรบอกตรงๆ ว่ามีช่องว่างอะไรบ้าง

สิ่งที่มีในไฟล์แล้ว

- การตัดสินใจที่ล็อกแล้ว: Flutter, Firestore offline cache, สต็อกกลาง, Member เห็นเฉพาะออเดอร์ของตัวเอง, App Store
- Schema Firestore ทั้งหมด พร้อมเหตุผลของโครงสร้างสำคัญ
- รูปแบบการเขียนข้อมูลตอน offline (WriteBatch, increment) และข้อควรระวัง
- บทบาท Owner / Manager / Member และตารางสิทธิ์
- Cloud Functions 5 ตัวและหน้าที่
- Repository layer ที่ต้องสร้างแทน Hive
- Acceptance criteria สำหรับ rules, offline, หลายร้าน, คลังกลาง และ Settlement
- ลำดับการสร้าง 9 ขั้น

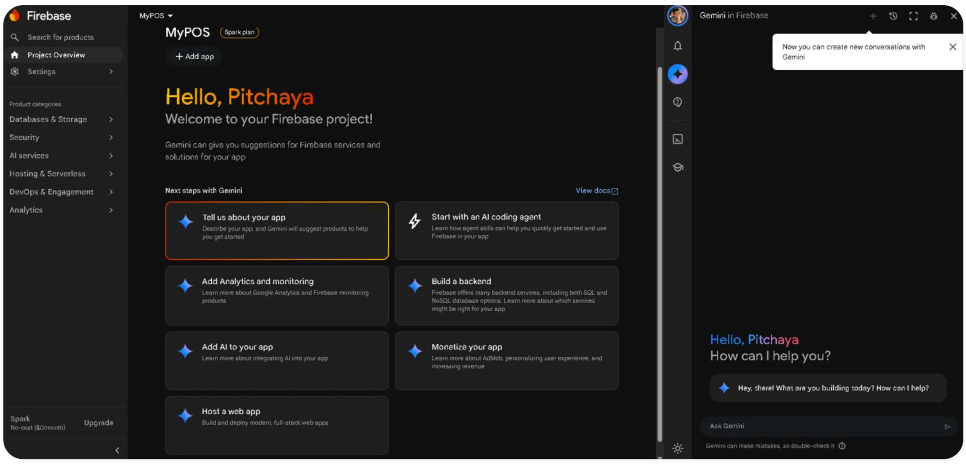
สิ่งที่ยังขาด

1. **ไฟล์ rules จริงยังไม่ได้แยกออกมา** ตอนนี้ `firestore.rules` และ `storage.rules` อยู่ในเอกสารเป็นข้อความ ยังไม่ได้ทดสอบด้วย Emulator จึงยังไม่รู้ว่าทำงานถูกต้อง
2. **ยังไม่มีไฟล์ `firestore.indexes.json` และ rules test**
3. **Sign in with Apple ยังไม่อยู่ในแพ็กเกจและขั้นตอนตั้งค่า** ทั้งที่จำเป็นสำหรับ App Store
4. **ยังไม่ได้เลือก Unlisted หรือ Public**
5. **ยังไม่มีตัวอย่างโค้ด Flutter** สำหรับ `placeOrder`, `voidOrder` และ repository ซึ่งเป็นจุดที่ AI มักทำผิดได้ง่ายที่สุด
6. **ยังไม่ได้ตรวจโค้ดจริงใน repo** ข้อมูลเกี่ยวกับ repo มาจาก README เท่านั้น

ควรให้ Antigravity ทำอย่างไร

- **เริ่มจากขั้นที่ 1 ถึง 2 ได้เลย** (ตั้งค่า Firebase, ล็อกอิน, สร้างร้าน) เพราะไม่ขึ้นกับ rules
- **ก่อนทำขั้นที่ 3 ควรมีไฟล์ rules และ rules test ก่อน** และต้องผ่าน Emulator ทุกข้อในหัวข้อ 12.1
- **ให้ตรวจโค้ดใน repo ก่อนเริ่ม** ว่าโครงสร้างตรงกับที่ README อ้างหรือไม่ เช่น โฟลเดอร์ `booth_pos_app/` ที่ยังไม่เห็นในรายการไฟล์ระดับ root

ถ้าต้องการ ผมเขียนไฟล์ `firestore.rules`, `storage.rules`, `firestore.indexes.json`, ชุด rules test และตัวอย่าง repository สำหรับ `placeOrder` / `voidOrder` แยกออกมาให้ได้เลย รวมถึงเพิ่ม Sign in with Apple ในแผน ส่วนการเลือก Unlisted หรือ Public ต้องรอคำตอบจากคุณ



ตอนนี้ใช้ firebase อยากให้นำส่วนที่ต้องจัดการใน firebase เขียนแล้วส่งต่อให้ Gemini Ai ทำอย่างละเอียด

Searched the web

Paused: Your free usage returns at 5:10 AM. Continue then. Continue